Restrição do Tamanho de Kernel em CNNs para Aceleração Winograd

Rafael Silva de Souza
Curso de Engenharia de Computação
Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
rafael.souza@inf.ufrgs.br

Mateus Grellert da Silva
Professor Adjunto, Orientador
Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
mateus.grellert@inf.ufrgs.br

***Abstract*—Aceleradores baseados no algoritmo de Winograd oferecem ganhos expressivos de eficiência para convoluções de kernel pequeno (2×2, 3×3) [2], mas escalam mal para kernels grandes (5×5, 11×11), comuns em arquiteturas clássicas como a AlexNet. Este trabalho investiga o custo de restringir o tamanho de kernel de uma CNN e se esse custo pode ser recuperado sem reintroduzir kernels grandes. Treinamos variantes da AlexNet no Tiny ImageNet-200 (64×64, 200 classes) sob um pipeline FP32 → *Quantization-Aware Training* (QAT) → INT8, comparando um baseline de kernel irrestrito, uma restrição ingênua a 3×3 e uma família de mecanismos de compensação estrutural leve — blocos *bottleneck* (1×1 → 3×3 → 1×1), módulos *Fire* [4] e um atalho residual isolado — todos mantendo os kernels ≤3×3. A variante com *bottleneck* (`AlexNetBottleneck`) atinge 44,62% de acurácia top-1 em FP32 com apenas 0,385M parâmetros (1,51 MB), mais de 4 pontos percentuais acima da variante que só troca os kernels por 3×3 (com *head* GAP), e com 6× menos parâmetros. Entre as CNNs que perdem acurácia na quantização INT8, é a que menos perde (−0,08 p.p.). Um experimento seguinte isola o efeito de um único atalho residual adicionado a cada bloco *Fire*, sem nenhum parâmetro extra (`AlexNetFireBypass`). Esse atalho sozinho rende, em FP32, mais que uma arquitetura híbrida bem mais complexa, que combina *stem* novo e *Fire* com atalho residual (ganho de 6,59 vs. 5,81 p.p. sobre o *Fire* de base), e continua à frente em INT8 (49,86% vs. 49,20%), embora perca 0,71 p.p. na quantização. Uma ressalva importante: o `AlexNetFireBypass` treinou por muito mais épocas (152 vs. 71). Medições diretas de hardware (GPU NVIDIA RTX 4060, rastreamento de kernel CUDA em nível de camada) indicam que a aceleração Winograd depende de convoluções densas (não agrupadas), não apenas do tamanho do kernel, e que compete com *tensor cores* em lotes maiores. Estendemos essas descobertas a duas frentes adicionais. Primeiro, testamos se a compensação estrutural e sua robustez à quantização transferem para predição densa — detecção SSD e segmentação DeepLabV3 em PASCAL VOC, com os três mesmos *backbones*. A transferência é parcial e depende do mecanismo: `AlexNetBottleneck` lidera detecção (mAP 20,98%) e permanece estável sob INT8 em ambas as tarefas, mas `AlexNetFire` — quase indistinguível de `AlexNetBottleneck` em classificação — cai para menos da metade do mAP (8,94%). Paridade em classificação não garante paridade em predição densa. Segundo, avaliamos se atenção local (janelas ao estilo Swin, ViT/DeiT) reproduz o perfil de acurácia/eficiência/quantização das CNNs de kernel pequeno, em sete modelos. Janelas menores custam acurácia, como kernels menores, e nenhum dos sete aciona um *kernel* Winograd em nenhuma configuração — consistente com a ausência de convoluções 3×3 de *stride* 1 nessas arquiteturas. Cinco dos sete modelos, porém, *ganham* acurácia sob quantização INT8 (até +2,10 p.p.), enquanto os outros dois caem apenas ligeiramente (−0,22 e −0,06 p.p.); a destilação DeiT é o mais forte dos sete (46,38% FP32), mas à custa de 5–7× mais parâmetros (2,76M) que as arquiteturas Pareto-ótimas deste estudo. Os resultados sugerem que compensação estrutural leve — e não kernels maiores — é uma via eficiente para recuperar acurácia em arquiteturas compatíveis com Winograd neste protocolo, e que mecanismos de custo zero (um único atalho residual) podem aproximar-se de arquiteturas híbridas mais elaboradas.**

***Index Terms*—Winograd, convolução, quantização, redes neurais convolucionais, kernel bottleneck, módulo Fire, atalho residual, INT8, detecção de objetos, segmentação semântica, atenção local, transformers de visão**

I. Introdução

O algoritmo de convolução de Winograd [2] reduz o número de multiplicações necessárias para convoluções de kernel pequeno, sendo a base de aceleradores de inferência amplamente usados em hardware embarcado. Sua vantagem, porém, desaparece rapidamente conforme o kernel cresce: o fator de redução de multiplicações frente à convolução direta satura numa constante em vez de crescer indefinidamente, e as matrizes de transformação tornam-se numericamente mal-condicionadas para kernels grandes [3], tornando kernels grandes (5×5, 11×11) pouco atrativos nesses aceleradores.

Isso cria uma tensão direta com arquiteturas clássicas de visão computacional. A AlexNet [1], por exemplo, depende de kernels 11×11 e 5×5 em suas primeiras camadas para capturar contexto espacial amplo. Restringir ingenuamente esses kernels a 2×2 ou 3×3 — sem qualquer outra mudança arquitetural — reduz potencialmente a capacidade do modelo, pois cada camada enxerga uma vizinhança muito menor da imagem.

Este projeto investiga se essa perda de acurácia pode ser recuperada por meio de mecanismos de *compensação estrutural* que mantêm todos os kernels compatíveis com Winograd (≤ 3×3), em vez de simplesmente devolver kernels grandes ao modelo. A hipótese central é que a restrição de tamanho de kernel, ainda que estruturalmente limitante, pode ser compensada por reorganizações arquiteturais de baixo custo — blocos *bottleneck*, módulos *Fire*, atalhos residuais — que redistribuem o poder representacional sem reintroduzir kernels grandes. Este relatório apresenta os resultados dessa investigação para dois mecanismos de compensação: blocos

bottleneck no estilo ResNet [5] (redução de canais 1×1 → convolução 3×3 → expansão 1×1) e módulos *Fire* no estilo SqueezeNet [4] (*squeeze* 1×1 → *expand* 1×1/3×3), aplicados a uma AlexNet com kernels restritos a 3×3. Em seguida, um experimento separa, dentro da arquitetura híbrida que combina *Fire* e atalhos residuais, o componente responsável pela maior parte do ganho de acurácia: um único atalho residual, sem nenhum parâmetro adicional.

Os resultados se organizam em sete eixos: custo da restrição de kernel e mecanismos de compensação (Eixo 1), combinações de mecanismos (Eixo 2), validação em hardware (Eixo 3), o atalho residual isolado (Eixo 4), robustez à quantização e compressão (Eixo 5), transferência para detecção e segmentação (Eixo 6) e atenção local (Eixo 7).

II. Metodologia

A. Dados e configuração experimental

Todos os modelos foram avaliados no Tiny ImageNet-200 [9] (imagens 64×64 RGB, 200 classes, *split* 90/10 determinístico) sob o mesmo pipeline de três estágios: (i) treino FP32 — do zero para a grande maioria das variantes, exceto a baseline AlexNet irrestrita e os baselines externos ResNet18 [5]/MobileNetV2 [6] (Seção III-A), que partem de pesos pré-treinados em ImageNet-1K e são apenas ajustados (*fine-tuned*) para as 200 classes do Tiny ImageNet; (ii) *Quantization-Aware Training* (QAT), inicializado a partir do melhor checkpoint FP32, com fusão Conv-BN-ReLU e *fake quantization* nos módulos suportados [7] — as arquiteturas com atalho residual usam um operador de soma dedicado à quantização no lugar da soma direta, requisito do PyTorch para que o *add* também seja quantizado corretamente; (iii) conversão para INT8 (*backend* fbgemm, inferência em CPU). Reportamos acurácia top-1 tanto para o modelo FP32 quanto para o modelo INT8 final, e a diferença INT8 − FP32 (ΔQAT, negativa quando a quantização custa acurácia) como medida de robustez à quantização.

O projeto foi executado em fases numeradas, e algumas legendas citam a fase de origem de cada modelo: Fases 1–4 (referências, restrição de kernel, compensação e arquiteturas híbridas, base dos Eixos 1, 2 e 5), Fase 6 (perfilamento de hardware, Eixo 3), Fase 7 (detecção e segmentação, Eixo 6), Fase 8 (atenção local, Eixo 7) e Fase 9 (atalho residual, Eixo 4). A Fase 11, citada em alguns pontos, é um experimento suplementar: retreina variantes da AlexNet por 500 épocas, sem *early stopping*, para separar o efeito do kernel do efeito da geometria da rede.

B. Arquiteturas comparadas

Sete variantes permitem contrastar o efeito da restrição de kernel (com as ressalvas de geometria descritas após a lista), o efeito do *head* GAP, o efeito de diferentes mecanismos de compensação, e o efeito de cada componente dentro de uma arquitetura híbrida. Todas são treinadas do zero sob o mesmo protocolo (Seção II-A), com exceção da baseline irrestrita, que é pré-treinada (ver abaixo):

- **AlexNet (irrestrita, `AlexNetTV`)** — a AlexNet original da torchvision, sem modificação na arquitetura além da camada de saída para 200 classes, com kernels de até 11×11, *pré-treinada em ImageNet-1K* e ajustada (*fine-tuned*) para as 200 classes; é a referência interna do experimento. Diferentemente de todas as demais variantes abaixo — treinadas do zero — essa baseline parte de pesos pré-treinados; comparações diretas de acurácia com ela confundem o efeito da restrição de kernel com o efeito do pré-treino (ver Seção IV). A Figura 1 mostra também a mesma arquitetura treinada do zero, no experimento suplementar da Fase 11.
- **AlexNet3x3-FC** — mesmos canais da AlexNet irrestrita, todos os kernels convolucionais restritos a 3×3, treinada do zero, mantendo a *head* totalmente conectada original. Difere da baseline não só no kernel, mas também em pré-treino, geometria de *stride*/*pooling* (adaptada a 64×64, ver abaixo), ausência de *dropout* e duração de treino; portanto a comparação com a baseline não isola o efeito do kernel. O par `AlexNet3x3-FC` vs. `AlexNet3x3-GAP` isola o efeito do *head*.
- **AlexNet3x3-GAP** — mesmo *backbone* da `AlexNet3x3-FC`, com *head* de *global average pooling* (GAP) [16] no lugar de camadas totalmente conectadas.
- **AlexNetBottleneck** — kernels igualmente restritos a 3×3, mas cada estágio é substituído por um bloco *bottleneck* (1×1 → 3×3 → 1×1, razão de redução 4×) seguido de GAP.
- **AlexNetFire** — kernels restritos a 3×3, cada estágio substituído por um módulo *Fire* [4] (*squeeze* 1×1 → *expand* 1×1 + 3×3, concatenados) seguido de GAP.
- **AlexNetFinalFireResidual** — arquitetura híbrida final que combina o módulo *Fire* acima com um novo *stem* 3×3 *stride*-2 e atalhos residuais envolvendo cada estágio *Fire*.
- **AlexNetFireBypass** — variante de controle que parte do `AlexNetFire` original (mesmo *stem*, mesmos módulos *Fire*) e adiciona apenas um atalho residual por identidade em cada estágio, sem nenhum parâmetro extra em relação ao `AlexNetFire` base. Separa o efeito do atalho residual do efeito do *stem* novo do `AlexNetFinalFireResidual`.

Geometria a 64×64. A AlexNet original (e a baseline pré-treinada) foi desenhada para entradas de 224×224: *stem* 11×11 com *stride* 4 e três *max-pool* 3×3 de *stride* 2 levam o mapa espacial a 6×6 antes do classificador. A 64×64 essa mesma geometria produz mapas de 15→7→3→1: o classificador recebe um mapa 1×1, que o *AdaptiveAvgPool* 6×6 apenas replica. As variantes treinadas do zero usam por isso uma geometria adaptada: *stem* com *stride* 2 e dois *max-pool* 2×2 (64→32→16→8), sem o terceiro *pool* e sem *dropout*. `AlexNetFire`, `AlexNetFireBypass` e `AlexNetSmallKernel` omitem o *stride* do *stem* e chegam a mapas de 16×16 antes do GAP, o que contribui para

seus MACs mais altos. A diferença importa: num experimento suplementar (Fase 11; 500 épocas sem *early stopping*, inicialização He, *head* GAP, sem BN, mesmos canais, semente única), a geometria adaptada (`AlexNetMixed`, kernels 3-2-3-2-3) atinge 45,28% de acurácia FP32, contra 27,90% com a geometria original (kernels 2-3-2-3-2), $\approx$17 p.p. a menos. Duas consequências: (i) a baseline irrestrita e as variantes restritas diferem em geometria, *dropout*, pré-treino e duração de treino, e não apenas no tamanho do kernel; (ii) os valores absolutos deste estudo são de 64×64 e não se extrapolam para AlexNets de 224×224 — o objeto de conclusão são as tendências qualitativas, não os números.

O bloco *bottleneck* e o módulo *Fire* preservam o campo receptivo efetivo da convolução 3×3 central, mas usam convoluções 1×1 para comprimir e depois expandir (ou concatenar) canais, aumentando a capacidade representacional por parâmetro sem introduzir nenhum kernel maior que 3×3 — mantendo a arquitetura inteiramente compatível com aceleração Winograd. O atalho residual por identidade usado no `AlexNetFireBypass` não introduz nenhuma convolução adicional, preservando trivialmente essa compatibilidade com Winograd.

Os Eixos 6 e 7 (Seções III-F e III-G) estendem a comparação a dois contextos; as arquiteturas usadas neles são descritas a seguir.

Predição densa (Eixo 6). Reaproveita, sem alterações no *backbone*, três das variantes acima — `AlexNetBottleneck`, `AlexNetFire` e `AlexNetTV` (a baseline de kernel irrestrito) — acopladas a uma *head* SSD [17] (detecção) ou DeepLabV3 [18] (segmentação); apenas a *head* muda entre classificação e predição densa, o *backbone* é idêntico ao já descrito.

Atenção local (Eixo 7). Sete arquiteturas testam se atenção local em janelas reproduz, em atenção, o perfil de restrição de kernel deste estudo:

- **swin_pico_{w2,w4,w8}** — Swin Transformer [22] em miniatura, dimensionada para entrada 64×64 (tamanho de *patch* 4), com tamanho de janela de atenção local $\in \{2, 4, 8\}$ — o análogo, em atenção, do tamanho de kernel convolucional.
- **swin_pico_poolmixer** — mesma arquitetura acima, mas com a atenção substituída por um *pooling mixer* simples (sem mecanismo de atenção real); controle para isolar se o ganho observado vem da atenção em si ou apenas da arquitetura ao redor dela.
- **hybrid_bottleneck_swin** — *stem* convolucional que reaproveita o bloco *bottleneck* desta seção para o rebaixamento espacial inicial, seguido de estágios de atenção em janela (Swin) para a mistura de contexto de longo alcance.
- **vit_tiny** — *Vision Transformer* (ViT) [21] compacto: a imagem é dividida em *patches* de tamanho fixo, cada *patch* é projetado linearmente em um *token*, e camadas de atenção multi-cabeça *global* padrão (não em janela — cada *token* atende a todos os demais, ao contrário do Swin acima) misturam informação entre todos os *tokens* simultaneamente, sem nenhuma convolução ou viés indutivo espacial. Dimensionado para 64×64 e treinado do zero com entropia cruzada padrão.
- **deit_tiny** — mesma arquitetura de `vit_tiny`, mas treinada com o esquema de destilação do DeiT (*Data-efficient image Transformer*) [23]: como transformers de visão carecem do viés indutivo convolucional e tipicamente exigem grandes volumes de dados ou pré-treino em larga escala para igualar CNNs, o DeiT treina o transformer para imitar diretamente os rótulos previstos (não apenas os rótulos verdadeiros) de um professor CNN já treinado — aqui, o `MobileNetV2` pré-treinado e congelado (*baseline* externo do Eixo 1, Figura 1) — reduzindo a dependência de grandes quantidades de dados de treino.

Por não haver caminho INT8 estável para LayerNorm/*softmax*/atenção neste *pipeline* de QAT (Seção II-A), essas camadas permanecem em FP32 durante a conversão nos sete modelos acima — apenas as camadas Linear/Conv convertem para INT8.

C. Convenção de tamanho de modelo

Todos os tamanhos em MB deste trabalho medem *apenas os pesos* — o dicionário de pesos serializado do modelo — tanto em FP32 quanto em INT8, de modo que as duas colunas sejam diretamente comparáveis.

O tamanho FP32 é obtido a partir do *checkpoint* de treino, que também carrega o estado do otimizador AdamW, o *scheduler* e o histórico de treino junto aos pesos: extraem-se apenas os pesos do modelo, que são serializados isoladamente. O tamanho INT8 é medido diretamente sobre o artefato convertido, que já é nativamente apenas pesos. Onde o *checkpoint* de treino original não sobreviveu, o tamanho FP32 foi recalculado de forma exata reconstruindo a arquitetura a partir do registro de modelos do projeto — o tamanho dos pesos serializados depende só da arquitetura, não dos pesos treinados.

A razão de compressão FP32→INT8 medida sob essa convenção tem mediana $3{,}92\times$ e máximo $3{,}99\times$, dentro do limite teórico de $4\times$ para a redução de 32 para 8 bits por peso. Os tamanhos de Eixo 6 (detecção/segmentação) usam o tamanho real medido do modelo efetivamente implantado (que exclui pesos do classificador pré-treinado original nunca usados no *forward pass* de detecção/segmentação), calculado separadamente.

D. Perfilamento de hardware

As medições de latência (Eixo 3) foram feitas em uma única GPU NVIDIA RTX 4060 Laptop, em uma única sessão de coleta, cronometrando camadas convolucionais isoladas com sincronização explícita de GPU antes e depois de 200 iterações cronometradas (50 iterações de *warmup* descartadas). O algoritmo de convolução (Winograd, direto, ou GEMM via *tensor cores* TF32) é escolhido internamente pelo cuDNN a cada configuração — não é forçado pelo experimento — e identificado *a posteriori* por rastreamento do nome do *kernel* CUDA executado via *profiling* nativo do PyTorch. Como é uma

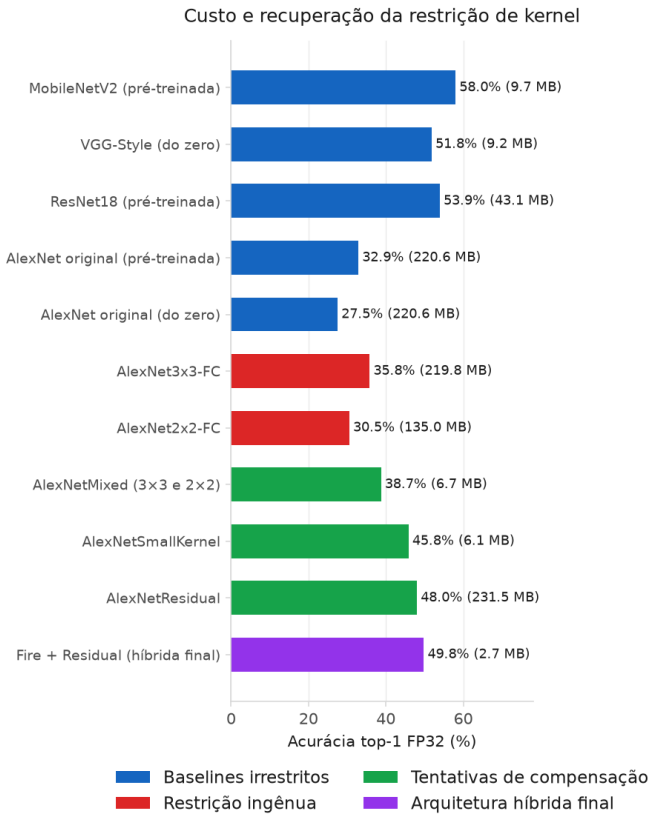


Figura 1. Acurácia FP32 (top-1) por arquitetura, com o tamanho dos pesos entre parênteses. Em azul, as referências de kernel irrestrito: MobileNetV2 e ResNet18 pré-treinadas, VGG-Style treinada do zero e a AlexNet original (`AlexNetTV`, sem modificação além da camada de saída) em duas versões, pré-treinada no ImageNet-1K e treinada do zero. Em vermelho, a restrição ingênua (todos os kernels trocados por 3×3 ou 2×2, sem outra compensação); em verde, tentativas exploratórias de compensação (`AlexNetMixed` e `AlexNetResidual` não são detalhadas individualmente neste relatório; `AlexNetSmallKernel` volta no Eixo 5); em roxo, a arquitetura híbrida final. A AlexNet do zero vem do experimento suplementar da Fase 11 (500 épocas sem *early stopping*, mesma receita AdamW); as demais barras seguem o protocolo da Seção II-A, com *early stopping*. Com a receita única adotada depois, no estudo das Fases 11–13 (SGD, medida no conjunto de teste), a mesma AlexNet treinada do zero chega a 35,5%.

GPU de notebook com limite de potência, o *clock* sustentado varia entre sessões (deriva de até 2× em medições de controle no mesmo dia), e todas as medições vêm de uma única sessão; diferenças pequenas de latência devem ser lidas com essa incerteza em mente (ver limitações na Seção IV). As medições INT8 rodam na CPU do mesmo notebook, porque o PyTorch só executa modelos INT8 em CPU.

III. Resultados

A. Eixo 1: Custo de Restrição de Kernel e Mecanismos de Compensação

A Figura 1 resume este eixo. Os três *baselines* externos situam o problema fora do escopo compatível com Winograd ($\leq$3×3) deste estudo: MobileNetV2 pré-treinada atinge a maior acurácia do gráfico, 58,0% em FP32 (9,69 MB); ResNet18 pré-treinada, 53,9% (43,07 MB, o maior dos três); e VGG-Style, treinada do zero, 51,8% (9,21 MB). A AlexNet original fica bem abaixo deles nas duas versões: 32,9% pré-treinada e 27,5% treinada do zero, mesmo com 500 épocas. A 64×64, sua geometria reduz o mapa espacial a 1×1 antes do classificador (Seção II-B), o que pesa mais que o tamanho do kernel. Do lado da restrição ingênua, `AlexNet3x3-FC` (35,79%, discutida abaixo) e `AlexNet2x2-FC` (30,53%) superam a AlexNet original treinada do zero, mas usam a geometria adaptada; com kernel 2×2 e sem compensação estrutural, a rede fica abaixo da AlexNet pré-treinada.

A Tabela I resume o custo de restringir kernels a 3×3 e a efetividade de dois mecanismos de compensação: blocos *bottleneck* e módulos *Fire*. Parâmetros e MACs medem eficiência de armazenamento e de computação separadamente — como o Eixo 3 mostra, eles nem sempre concordam sobre qual arquitetura é "mais eficiente". `AlexNet3x3-FC` mantém a *head* totalmente conectada da baseline e restringe kernels a 3×3, mas também treina do zero (a baseline é pré-treinada) e usa a geometria adaptada a 64×64 (Seção II-B), portanto não isola o efeito do kernel. Note que, apesar do kernel menor, `AlexNet3x3-FC` tem mais MACs que a baseline (222,3M vs. 94,9M): o *stem* 3×3 usa *stride* 2 em vez do *stride* 4 do *stem* 11×11 original, então os mapas de ativação permanecem maiores por mais camadas — um kernel menor por aplicação, mas aplicado sobre uma área espacial bem maior no total.

A `AlexNet3x3-FC` (35,79%) supera a baseline irrestrita (32,89%) em 2,9 p.p., ambas com *head* totalmente conectada. Esse resultado *não* pode ser lido como um ganho causado pela restrição de kernel: as duas diferem também em pré-treino, em geometria (a baseline preserva o *stride* 4 e os *pools* 3×3 de 224×224, que a 64×64 reduzem o mapa a 1×1 antes do classificador; Seção II-B), em *dropout* (só a baseline o usa) e em duração de treino (79 vs. 37 épocas). O que os dados sustentam é mais fraco: uma AlexNet de kernels 3×3, treinada do zero com geometria adaptada, iguala ou supera a baseline pré-treinada em 64×64. O estudo não contém a variante que isolaria o kernel — a mesma geometria adaptada com kernels 11×11/5×5/3×3. O único controle de kernel a geometria constante (Fase 11: AlexNetTV do zero, geometria original, 500 épocas, semente única) deu −1,0 p.p. ao restringir a 3×3 e −2,5 p.p. a 2×2 (27,51%→26,50%→25,03%; o primeiro valor é a barra "AlexNet original (do zero)" da Figura 1), com ressalvas: o modelo de kernels originais usou outra inicialização, e com *stride* 4 o *stem* 3×3 (2×2) lê apenas 56% (25%) dos pixels de entrada, contra 100% do 11×11. Uma explicação plausível, entre outras, para a diferença residual é o *overfitting* da *head* FC de 57M parâmetros da baseline num dataset 64×64; isso também não foi testado.

A mudança de efeito isolado mais claro é a troca de *head* para GAP (`AlexNet3x3-GAP`): mais 4,5 p.p. (35,79%→40,32%) com 25× menos parâmetros (57,6M → 2,30M, quase todos eliminados da *head* FC), consistente com redução de *overfitting* dada a dimensão reduzida do dataset.

A compensação estrutural acrescenta ganhos além da troca de *head*. O bloco *bottleneck* soma mais 4,3 p.p. sobre a variante GAP com 6× menos parâmetros (ressalva: o *bottleneck*

Tabela I
CUSTO DE RESTRIÇÃO DE KERNEL (3×3) E MECANISMOS DE COMPENSAÇÃO

Modelo	**Params (M)**	**MACs (M)**	**Tam. FP32 (MB)**	**Tam. INT8 (MB)**	**Top-1 FP32 (%)**	**Top-1 INT8 (%)**	**ΔQAT (p.p.)**
AlexNet (irrestrita, pré-treinada)	57,82	94,9	220,58	55,33	32,89	31,90	−0,98
AlexNet3x3-FC (do zero)	57,61	222,3	219,75	55,12	35,79	36,19	+0,40
AlexNet3x3-GAP	2,30	167,0	8,79	2,23	40,32	37,02	−3,29
AlexNetBottleneck	**0,39**	**39,5**	**1,51**	**0,43**	**44,62**	**44,54**	**−0,08**
AlexNetFire	0,52	177,7	2,01	0,55	43,98	44,30	+0,33

usa *BatchNorm* em todas as convoluções e lr 10^{-3}; a variante GAP não usa BN e treinou com 3×10^{-4}; ver Limitações), e praticamente elimina a perda sob INT8. O módulo *Fire* é um mecanismo alternativo: 0,52M parâmetros, 43,98% em FP32 e um *ganho* sob quantização (INT8 44,30%, +0,33 p.p.) — propriedade rara neste estudo, compatível com a hipótese de que a compressão *squeeze-expand* facilita a calibração de quantização, embora o mecanismo exato permaneça especulativo. Em MACs, porém, essa economia de parâmetros não se traduz em menos computação: o Fire usa 177,7M MACs, mais que a própria `AlexNet3x3-GAP` (167,0M) e $4,5\times$ mais que o *Bottleneck* (39,5M) — apenas o *Bottleneck* reduz armazenamento *e* computação.

B. Eixo 2: Sinergia de Arquiteturas Híbridas

Esta seção testa se dois mecanismos de compensação combinados rendem mais do que cada um sozinho. A Tabela II compara quatro combinações de *bottleneck*, *Fire*, atalhos residuais e convoluções *depthwise* com as duas arquiteturas base das quais elas derivam (`AlexNetFire` e `AlexNetBottleneck`, já vistas na Tabela I). Nas figuras e no apêndice, as quatro combinações aparecem como `AlexNetFinalFireResidual`, `AlexNetFinalBottleneckFire`, `AlexNetFinalBottleneckResidual` e `AlexNetFinalDepthwiseFire`, nesta ordem.

A combinação Fire + Residual é a mais efetiva, com ganho de 5,81 p.p. sobre o Fire de base (43,98% → 49,79%) — um efeito não linear que sugere sinergia entre compressão de canais (*squeeze-expand*) e conexões residuais. As combinações com *Bottleneck* saem piores: Bottleneck + Fire (42,29%) fica *abaixo dos dois* pais (Fire 43,98%, Bottleneck 44,62%), e Bottleneck + Residual (45,10%) supera o `AlexNetBottleneck` sozinho por apenas 0,48 p.p., apesar do atalho residual adicional — bem menos sinergia do que Fire + Residual obtém sobre seu próprio pai. Nem todos os mecanismos se combinam igualmente bem, e o *Bottleneck* em particular parece perder mais do que ganhar ao ser combinado.

Nesta comparação, parâmetros e MACs não acompanham a acurácia. Bottleneck + Fire tem menos parâmetros (0,51M) e menos MACs (47,3M) que o Fire de base (0,52M, 177,7M) e ainda assim *perde* 1,7 p.p. de FP32 (43,98%→42,29%) — o oposto do padrão do Eixo 1, em que os mecanismos de compensação isolados ganhavam acurácia junto com eficiência. Só a combinação Fire + Residual melhora nos dois eixos ao mesmo tempo (55,2M MACs, +5,81 p.p.); as demais trocam MACs por acurácia, não somam os dois.

A eficiência de Depthwise + Fire também é enganosa para o objetivo deste projeto. É o modelo com menos MACs da tabela (30,1M, ∼6× menos que o Fire de base), mas usa convoluções *depthwise* em seu *stem*, e o Eixo 3 mostra que o cuDNN nunca aciona Winograd para convoluções *depthwise*, em nenhum tamanho de kernel testado. A economia de MACs desse modelo, portanto, não se traduz em aceleração Winograd — ele é eficiente em MACs sem ser compatível com Winograd, uma distinção que este relatório trata como central.

O padrão de ΔQAT também segue o mecanismo, não a combinação. As duas combinações que incluem *Bottleneck* ganham acurácia sob INT8 (+1,71 e +0,88 p.p.), ecoando o achado do Eixo 1 de que o *Bottleneck* isolado "praticamente elimina a perda sob INT8" — essa estabilidade parece se propagar para os híbridos. Já o *Fire* muda de sinal conforme o parceiro: +0,33 p.p. sozinho (Tabela I), −0,59 com Residual, −0,67 com *Depthwise*.

C. Eixo 3: Validação em Hardware da Aceleração Winograd

Investigamos se as arquiteturas do projeto realmente acionam aceleração Winograd em GPU, e sob quais condições. Duas fontes de evidência, de força bem diferente, são usadas: (i) rastreamento direto do nome do *kernel* CUDA executado pelo cuDNN — a evidência mais forte, pois identifica o algoritmo em vez de inferi-lo; e (ii) o *scaling* de latência com o tamanho do kernel, testado estatisticamente — uma evidência indireta e, como mostrado abaixo, inconclusiva no regime que mais importa.

O rastreamento de *kernel* indica que o cuDNN seleciona um algoritmo identificado como Winograd especificamente para convoluções densas (não agrupadas) com kernel 3×3, nos *batches* 1 e 8 — mas não para convoluções *depthwise*, em nenhum tamanho de kernel (2 a 11) ou *batch* testado nesta GPU e backend. Esse é o resultado mais direto desta seção.

O teste de *scaling* teórico é mais fraco. A complexidade de Winograd não é assintoticamente menor que a da convolução direta: para um *tile* de saída fixo de $m \times m$ elementos, o número de multiplicações de Winograd, $(m+k-1)^2$, e o da convolução direta, m^2k^2, crescem ambos como $\Theta(k^2)$ com o tamanho do kernel k [2] — por exemplo, F(2×2,3×3) usa 16 multiplicações contra 36 da convolução direta $(2,25\times)$, e F(4×4,3×3) usa 36 contra 144 $(4\times)$ [2]; essa vantagem converge para um fator constante $(m^2\times)$ quando $k \to \infty$,

Tabela II
ARQUITETURAS HÍBRIDAS: COMBINAÇÕES DE DOIS MECANISMOS DE COMPENSAÇÃO

Arquitetura	**Params (M)**	**MACs (M)**	**Tam. FP32 (MB)**	**Tam. INT8 (MB)**	**Top-1 FP32 (%)**	**Top-1 INT8 (%)**	**ΔQAT (p.p.)**
AlexNetFire (base)	0,52	177,7	2,01	0,55	43,98	44,30	+0,33
AlexNetBottleneck (base)	0,39	39,5	1,51	0,43	44,62	44,54	−0,08
Fire + Residual	0,70	55,2	2,71	0,75	49,79	49,20	−0,59
Bottleneck + Fire	0,51	47,3	1,97	0,55	42,29	44,00	+1,71
Bottleneck + Residual	0,57	54,2	2,24	0,64	45,10	45,98	+0,88
Depthwise + Fire	0,47	30,1	1,85	0,51	43,46	42,79	−0,67

em vez de crescer indefinidamente. Na prática, Winograd raramente é usado além de $k = 3$ (ocasionalmente $k = 5$) não por essa escala assintótica, mas porque as matrizes de transformação ficam numericamente mal-condicionadas para k grande [3]. A Figura 2 mostra a latência por FLOP medida para $k \in \{2, 3, 5, 7, 9, 11\}$. Dois regimes importam aqui: o *compute-bound* (*batch* $= 64$, GPU saturada, tempo dominado por cálculo real — exatamente onde uma redução de multiplicações deveria aparecer, e por isso a evidência primária) e o *overhead-bound* (*batch* $= 1/8$, GPU ociosa, tempo dominado por setup de kernel e sincronização).

Um teste de Wilcoxon pareado[1] [10] sobre a latência por FLOP (k=3 vs. k=5, denso) no regime compute-bound é **estatisticamente significativo, mas na direção oposta à esperada**: $p = 0,023$, $n = 8$, e a mediana do custo por FLOP em k=3 não é menor que em k=5. O teste rejeita a hipótese nula de nenhuma diferença sistemática, mas não corrobora a hipótese de que Winograd é mais eficiente por FLOP nesse regime. No overhead-bound há uma tendência favorável (mediana ∼10,8% menor em k=3), sem significância estatística ($p = 0,82$, $n = 16$) — ali o overhead mascara qualquer ganho de redução de FLOP. Consistente com isso, o rastreamento de *kernel* mostra que em *batch* $= 64$ o cuDNN troca Winograd por um *kernel* TF32 de *tensor core* mesmo em k=3: no regime onde a redução de FLOP teria mais efeito, nem sempre há Winograd para medir. A leitura mais defensável não é que Winograd não ajuda, mas que *tensor cores* competem com a vantagem de Winograd justamente no regime *compute-bound*, onde uma redução de multiplicações teria mais efeito — achado interessante para o acelerador FPGA que motiva este projeto (sem *tensor cores*), mas que este experimento em GPU não pode confirmar diretamente.

Para convoluções *depthwise*, o quadro é mais consistente: nenhuma evidência de Winograd foi encontrada em nenhum regime. No nível de camada, o mesmo teste de Wilcoxon [10] (compute-bound, denso vs. *depthwise*) rejeita sinal Winograd para *depthwise* ($p = 0,0078$, $n = 8$). No nível de modelo completo, `AlexNetDepthwiseSep` e `MobileNetV2` (85,0 e 24,9 GFLOP/s, mediana 54,9) ficam bem abaixo de quatro modelos densos comparáveis (`AlexNetBottleneck`, `AlexNetFinalBottleneckResidual`, `AlexNetFinalFireResidual`, `VGG-Style`; mediana 112,5 GFLOP/s) — comparação de apenas $n = 2$ modelos *depthwise*, exploratória. O mais correto a afirmar é que, *nesta GPU e neste backend (cuDNN)*, convoluções *depthwise* não acionam um *kernel* Winograd, não que o algoritmo seja matematicamente incompatível com elas — a literatura descreve variantes de Winograd por canal para convoluções *depthwise* [13]; o que este experimento demonstra é que o cuDNN, nesta configuração, não as implementa.

Mapeamos também a fronteira de Pareto entre acurácia e latência para o conjunto completo de arquiteturas (Figura 3).

As cinco arquiteturas com convolução 3×3 densa desta figura (`AlexNetBottleneck`, `AlexNetFire`, `AlexNetFinalBottleneckResidual`, `AlexNetFinalFireResidual` e `VGG-Style`) superam o baseline (AlexNetTV, irrestrito) na razão acurácia/latência. A latência em FP32 também é um preditor razoável do *ranking* de latência em INT8, mas só para convoluções densas (correlação de Spearman $\rho = 0,99$, $n = 24$); para *depthwise* a correlação é sensivelmente mais fraca ($\rho = 0,72$, $n = 24$) — o *backend* INT8 (fbgemm) tem *kernels* otimizados só para 3×3/5×5 em convoluções agrupadas [8] e cai para um caminho genérico muito mais lento acima disso, o que quebra a correspondência FP32→INT8 nesse subconjunto. Ambas as correlações são medidas em nível de camada, com FP32 e INT8 cronometrados na mesma CPU; o equivalente em nível de modelo compara FP32 em GPU com INT8 em CPU ($\rho = 0,70$, $n = 10$) e vale apenas como exploração.

Por fim, avaliamos a competição com *tensor cores* entre tamanhos de *batch*, no mesmo experimento acima: em *batch*=1 e 8 o cuDNN prefere Winograd em k=3; em *batch*=64 prefere um *kernel* TF32 de *tensor core* mesmo em k=3. Isso é uma limitação da GPU como plataforma de validação, não uma medição do princípio em si: o acelerador FPGA que motiva este projeto não possui *tensor cores*, então essa competição específica não deveria ocorrer nele — mas essa é uma expectativa baseada na ausência de *tensor cores* no design do FPGA, não algo medido neste trabalho.

D. Eixo 4: Atalho Residual de Custo Zero

O Eixo 2 mostrou que a arquitetura híbrida Fire + Residual ganha 5,81 p.p. sobre o *Fire* de base. Esta seção verifica se esse ganho vem principalmente do atalho residual. Para

[1]Não-paramétrico, adequado para comparar duas condições medidas nas mesmas configurações sem assumir normalidade dos dados.

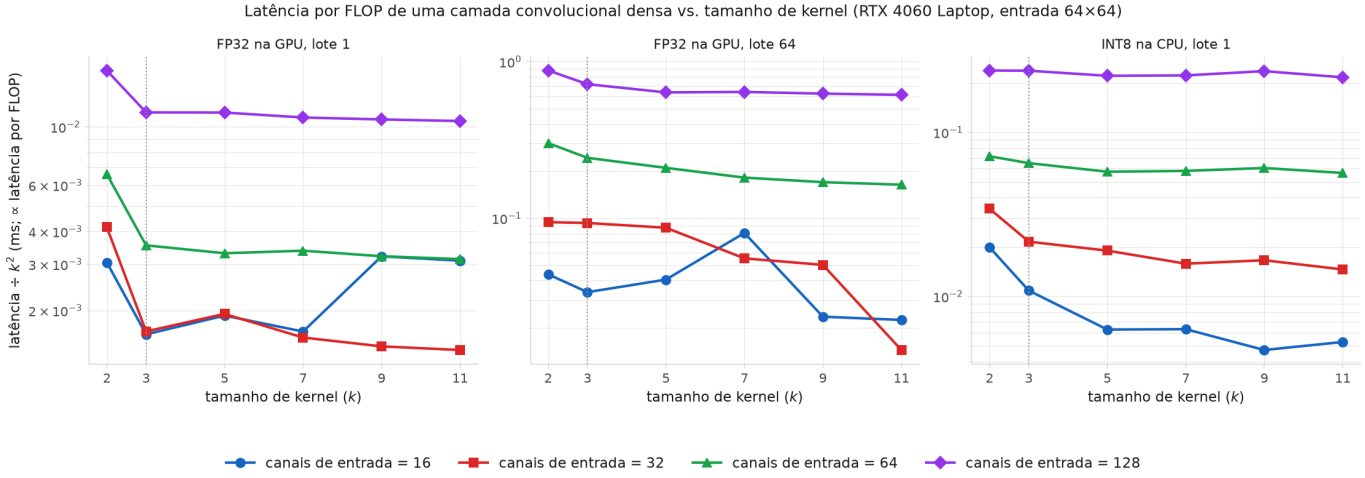


Figura 2. Latência de uma camada convolucional densa dividida por k^2 (proporcional à latência por FLOP) vs. tamanho de kernel k, para quatro números de canais de entrada: FP32 na GPU RTX 4060 Laptop com lote 1 e 64, INT8 na CPU com lote 1. A linha pontilhada marca $k = 3$, onde o cuDNN pode usar Winograd. A vantagem esperada em $k = 3$ aparece de forma inconsistente entre regimes — ver o texto para o teste estatístico e sua limitação no regime *compute-bound* (lote 64, painel central).

isso, adicionamos ao `AlexNetFire` original um atalho por identidade em cada estágio *Fire*, sem nenhum parâmetro extra e sem o *stem* novo da híbrida. A Tabela III compara o *Fire* de base, essa variante (`AlexNetFireBypass`) e a arquitetura híbrida completa.

Com os mesmos 0,52M parâmetros e 177,7M MACs do *Fire* de base, o atalho isolado ganha 6,59 p.p. em FP32, mais que os 5,81 p.p. da arquitetura híbrida completa, e também fica à frente dela em INT8 (49,86% vs. 49,20%). Na passagem para INT8, os dois perdem quantidades parecidas ($-0{,}71$ e $-0{,}59$ p.p.), enquanto o *Fire* de base chegava a ganhar ($+0{,}33$ p.p.): o atalho melhora a acurácia, mas não a robustez à quantização. O resultado sugere que o atalho residual, e não o *stem* novo, responde pela maior parte do ganho da arquitetura híbrida, e que ele pode ser adicionado sem custo em parâmetros.

Uma ressalva importante: o `AlexNetFireBypass` treinou por mais épocas que o restante do estudo (152 épocas efetivas, contra 71 do `AlexNetFinalFireResidual`; Tabela XI no apêndice), porque rodou no *cluster* PCAD da UFRGS, com orçamento de épocas maior que o disponível localmente. Parte do ganho pode, portanto, vir do treino mais longo, e não só do atalho. A Figura 4 situa esse resultado no conjunto completo de modelos do estudo.

A Figura 4 ordena todos os modelos de classificação do estudo segundo acurácia versus tamanho do modelo, complementando a visão latência-cêntrica da Figura 3. Quatro leituras se destacam.

Nem todo modelo de atenção na base da fronteira FP32 está lá pelo mesmo motivo. A fronteira é uma escada de 7 modelos, de `swin_pico_poolmixer` (0,89 MB, 30,98%) a `MobileNetV2` (9,69 MB, 58,00%). O primeiro degrau, `swin_pico_poolmixer`, está lá por motivo trivial — é o menor modelo do estudo, não um bom compromisso. Mas o segundo, `hybrid_bottleneck_swin` (+0,16 MB), é o degrau mais acentuado de toda a fronteira: +9,25 p.p. Esse salto sozinho já é maior que o salto final para `MobileNetV2` (+6,18 p.p.). `AlexNetDepthwiseSep` (+0,18 MB) soma outros +4,16 p.p., e o `AlexNetFireBypass` (+0,50 MB sobre `AlexNetBottleneck`) entra na fronteira com +5,95 p.p.

Sob INT8, quase nenhum modelo de atenção continua na fronteira. A fronteira encolhe para 5 modelos e só `swin_pico_poolmixer` (0,305 MB, 30,76%) permanece como ponto de atenção local. `hybrid_bottleneck_swin`, apesar do FP32 competitivo (40,23%, Eixo 7), é dominado por `AlexNetDepthwiseSep` (0,40 MB, 41,47%) — menor *e* mais acurado. Mesmo `deit_tiny` (46,38% FP32, o melhor dos sete modelos de atenção originais — superado apenas pela variante `vit_tiny_convstem`, 48,00%) fica dentro da fronteira, não sobre ela: seu tamanho de pesos (∼10,55 MB) é maior que o de `VGG-Style` (9,21 MB, 51,81%), que o domina em ambos os eixos.

O controle sem atenção fica entre os modelos mais fracos. `swin_pico_poolmixer` (30,98%) supera por pouco `AlexNet2x2-FC` (30,53%) e a reexecução curta de `AlexNet2x2-GAP` (27 épocas, 30,02%; a execução de 44 épocas usada em todo o resto do relatório atinge 33,15% — ver legenda) — `AlexNetSE` fica fora da figura (colapso de treino, não um ponto real de eficiência; ver legenda). Mas fica atrás de todas as outras variantes `swin_pico`, inclusive a mais restrita em janela (`swin_pico_w2`, 32,95%). A hipótese de que a atenção em si, e não só a arquitetura ao redor dela, contribui para a acurácia das demais variantes `swin_pico` permanece plausível, mas o efeito é modesto (cerca de 2 p.p. sobre a janela 2).

Os *baselines* pré-treinados lideram em FP32, mas ao custo de tamanho. `MobileNetV2` (9,69 MB) e `ResNet18` (43,07 MB) têm a melhor acurácia FP32, mas são de 6 a 28 vezes maiores que `AlexNetBottleneck` (1,51 MB) e não

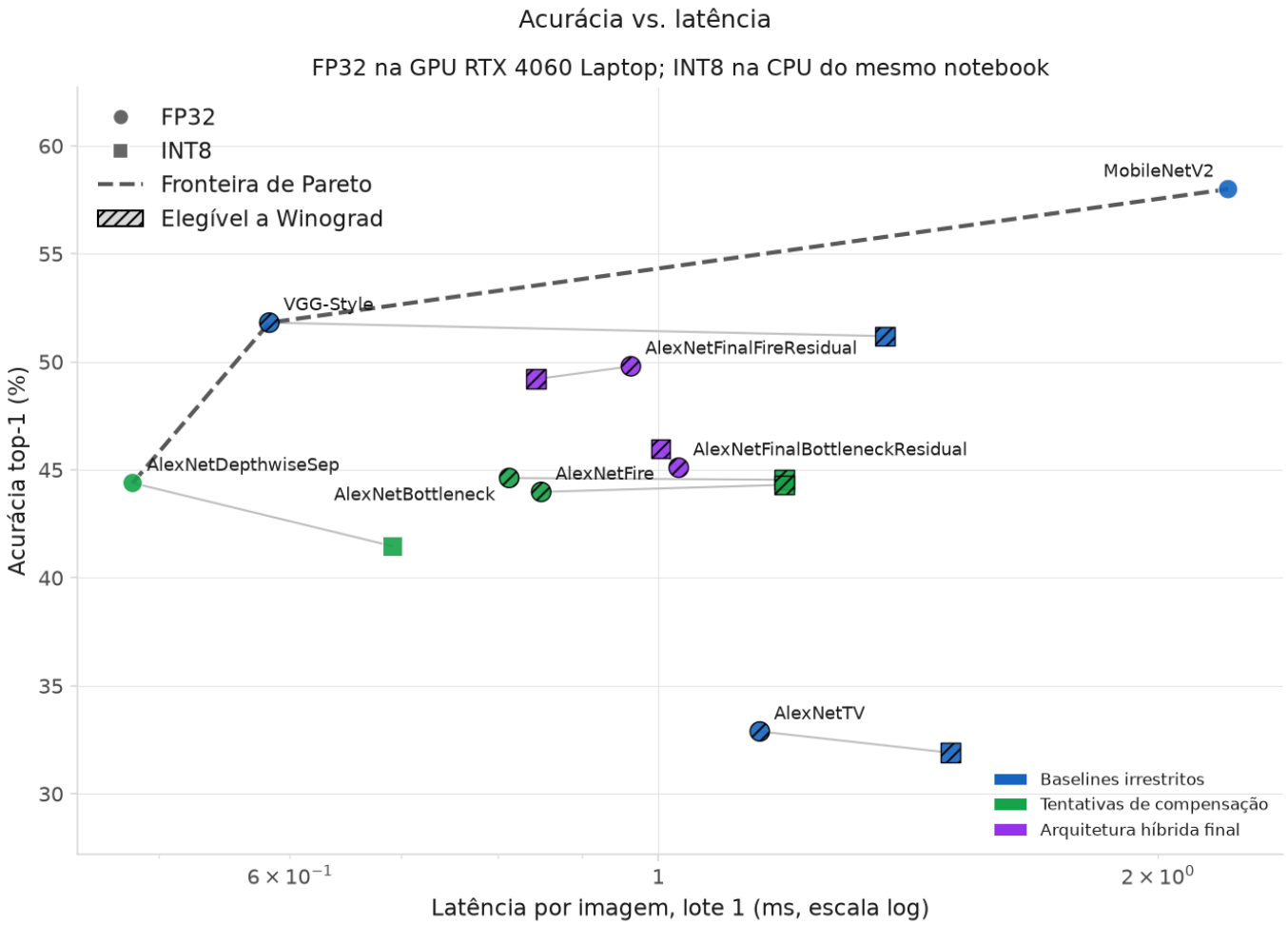


Figura 3. Acurácia × latência por imagem, com lote 1. FP32 (círculos) foi medido na GPU RTX 4060 Laptop; INT8 (quadrados), na CPU do mesmo notebook, porque o PyTorch só executa INT8 em CPU — as latências FP32 e INT8 não se comparam diretamente. Uma linha cinza liga as duas versões de cada modelo; a linha tracejada é a fronteira de Pareto. Cores como na Figura 1; hachura indica elegibilidade arquitetural a Winograd (há convolução 3×3 densa de *stride* 1), não o acionamento medido (Eixo 3), mesma convenção da Figura 4. MobileNetV2 e `AlexNetDepthwiseSep` não são elegíveis (convoluções *depthwise* e *pointwise*), e MobileNetV2 não tem ponto INT8 (não passou por QAT).

Tabela III
EFEITO ISOLADO DO ATALHO RESIDUAL: *Fire* DE BASE, *Fire* COM ATALHO E ARQUITETURA HÍBRIDA COMPLETA

Arquitetura	**Params (M)**	**MACs (M)**	**Top-1 FP32 (%)**	**Top-1 INT8 (%)**	**Ganho Vs. Fire (p.p.)**
AlexNetFire (base)	0,52	177,7	43,98	44,30	—
AlexNetFireBypass	**0,52**	**177,7**	**50,57**	**49,86**	**+6,59 FP32**
AlexNetFinalFireResidual (híbrida completa)	0,70	55,2	49,79	49,20	+5,81 FP32

têm ponto INT8 (não passaram por QAT).

Na Figura 5, a fronteira de Pareto FP32 tem só dois pontos: `swin_pico_poolmixer` (21,2M MACs, 30,98%), que entra apenas por ser o modelo de menor custo computacional desta figura — a mesma leitura já feita para o eixo de tamanho; no conjunto completo do estudo o mínimo é `AlexNetDepthwiseSep`, com 19,8M MACs — e `MobileNetV2` (25,8M MACs, 58,00%), que a domina de fato, com menos MACs que o próprio módulo *Bottleneck* (39,5M) e a melhor acurácia do estudo. Na fronteira INT8, mais povoada, os modelos de atenção não avançam além desse degrau inicial: `hybrid_bottleneck_swin` (30,3M MACs, 40,18%) fica de fora, dominado por `AlexNetFinalDepthwiseFire` (30,1M MACs, 42,79%) — menos MACs *e* mais acurácia.

O eixo MACs também reordena o agrupamento de compensação visível por tamanho na Figura 4: `AlexNetFire` e `AlexNetFireBypass` (177,7M MACs cada, já que o atalho por identidade não adiciona multiplicações) custam mais em computação

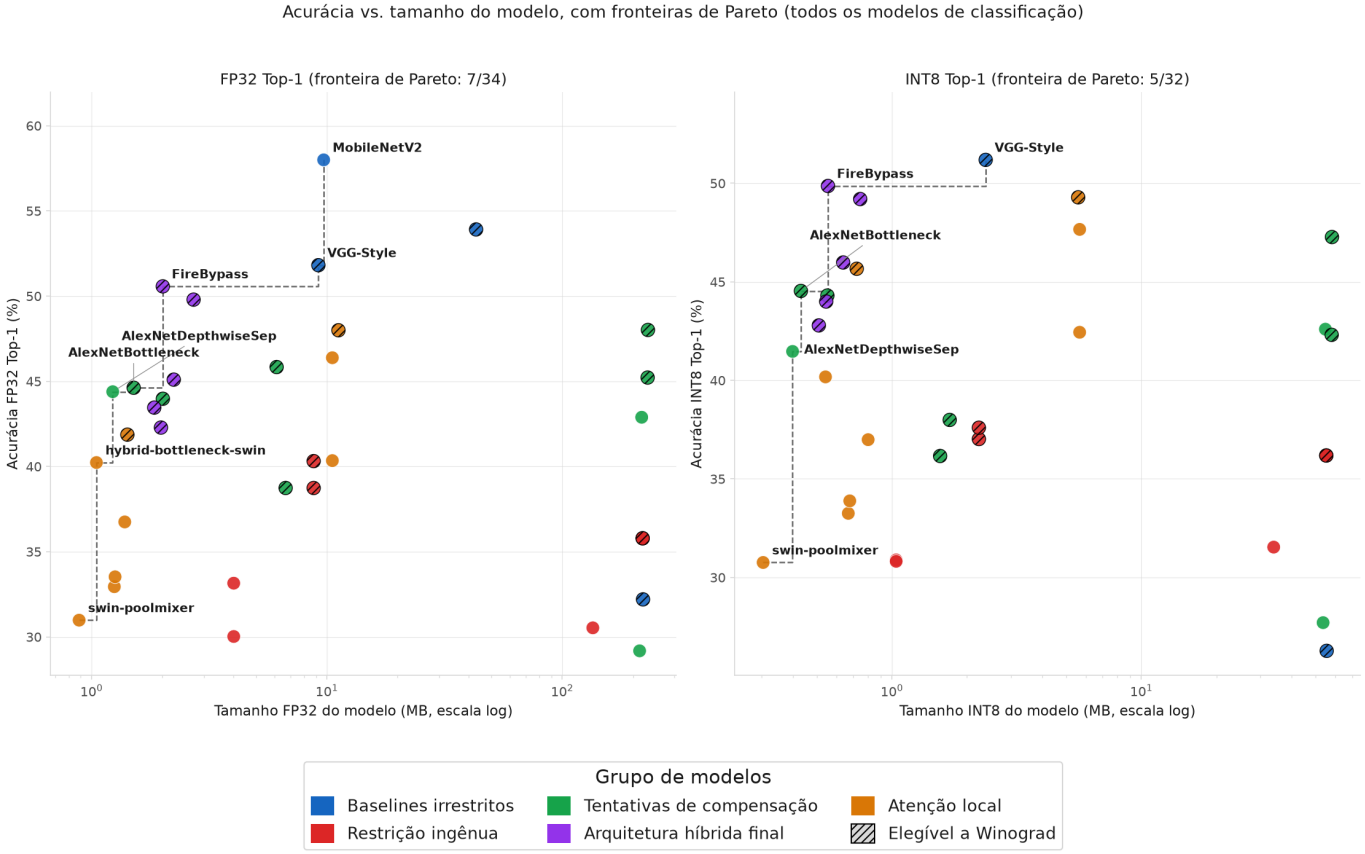


Figura 4. Acurácia top-1 vs. tamanho do modelo (pesos, MB, escala log) para os 34 modelos de classificação deste estudo (Fases 1–4, 8 e 9, incluindo as variantes com camada 3×3 no *stem* `vit_tiny_convstem`/`swin_pico_convstem`; omitidos: o Eixo 6, que mede mAP/mIoU, não top-1, e por isso não é comparável nesta mesma escala, e `AlexNetSE`, cujo treino malsucedido (∼0,5% top-1, Tabela XI) não é um ponto real de eficiência). Três dos 34 pontos (`alexnet_2x2`, `alexnet_3x3`, `alexnet_gap`) são reexecuções curtas superadas de arquiteturas já presentes na figura (`AlexNet2x2-GAP`, `AlexNet3x3-FC`, `AlexNet3x3-GAP`): mesma arquitetura e mesmo tamanho, acurácia menor ou igual (`alexnet_3x3` repete o mesmo resultado FP32 de `AlexNet3x3-FC`); nenhuma entra na fronteira. Dois painéis, FP32 (esquerda) e INT8 (direita), cada um com sua própria fronteira de Pareto tamanho-acurácia (linha tracejada; apenas os modelos na fronteira são rotulados, para legibilidade). Cor indica a fase de origem do modelo (mesma legenda em ambos os painéis); hachura indica elegibilidade arquitetural a Winograd (convolução 3×3 de *stride* 1 densa presente) — não o traço medido de acionamento real (Eixo 3/7, disponível só para um subconjunto dos modelos aqui). O painel INT8 tem 5 de 32 modelos na fronteira (contra 7 de 34 em FP32) e apenas 32 dos 34 modelos aparecem — MobileNetV2 e ResNet18 pré-treinados nunca passaram por QAT (*residual adds* sem operador de soma dedicado à quantização, Seção III-A).

que `AlexNetFinalFireResidual` (55,2M) e `AlexNetBottleneck` (39,5M), apesar de serem modelos bem menores — o mesmo padrão "poucos parâmetros, muitos MACs" do módulo *Fire* já discutido no Eixo 1.

E. Eixo 5: Robustez de Quantização e Comparação de Métodos de Compressão

Além de acurácia e latência, investigamos como diferentes mecanismos de compensação afetam a robustez sob quantização INT8 e compressão adicional.

O padrão geral: a compensação estrutural (bottleneck, Fire) com kernels restritos perde pouca acurácia na quantização, enquanto restrições ingênuas (AlexNet3x3-GAP, AlexNetSmallKernel) perdem bem mais. `AlexNetSmallKernel` é uma variante deliberadamente enxuta: mesmo *backbone* totalmente 3×3 do `AlexNet3x3-GAP`, mas com canais bem mais estreitos (64→128→256→256→256) e, como o `AlexNet3x3-GAP`, sem *BatchNorm*. Com 1,60M parâmetros é uma das variantes de kernel restrito sem compensação mais enxutas — atrás apenas do `AlexNet2x2-GAP` (1,05M) —, mas é a que mais sofre com a quantização em todo o estudo: sem BN após cada convolução e com canais estreitos, resta pouca margem de ativação para absorver o ruído de quantização — ao contrário de `AlexNetBottleneck`/`AlexNetFire`, que têm BN após toda convolução (Seção III-A). A Tabela IV agrupa por família as 13 variantes das Fases 2–3 com *checkpoint* INT8, mais o *baseline* irrestrito da Fase 1, e mostra o ΔQAT (*top-1* INT8 − FP32) de cada uma: o padrão não é uniforme dentro de cada família — `AlexNetDepthwiseSep` (−2,92 p.p.) cai tanto quanto variantes ingênuas medianas — mas o pior caso de compensação (−2,92) fica bem acima do pior caso ingênuo (`AlexNetSmallKernel`, −9,67), e a média por família (linha *Média* da tabela: compensação −0,86 p.p. vs. ingênua −2,51 p.p., quase 3× maior) confirma a tendência.

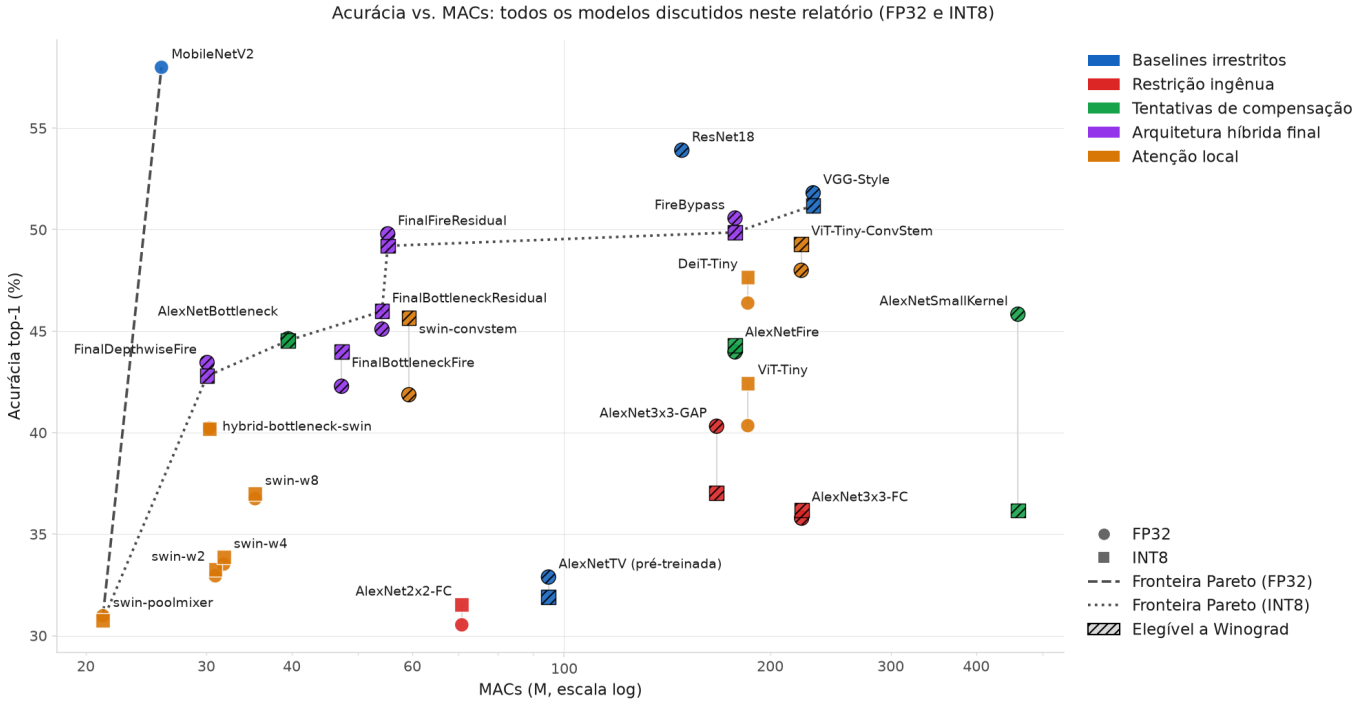


Figura 5. Acurácia vs. MACs (M, escala log) para 24 modelos — os 15 das Fases 1–4 e 9 mais os nove modelos de atenção local do Eixo 7, incluindo as variantes com camada 3×3 no *stem* `vit_tiny_convstem`/`swin_pico_convstem` — do conjunto completo comparado por tamanho na Figura 4. MACs não muda entre FP32 e INT8 — mesma arquitetura, mesma contagem de multiplicação-acumulação — por isso os pontos FP32/INT8 de cada modelo compartilham a mesma posição no eixo X, unidos por uma linha vertical em vez da diagonal usada na Figura 4. Fronteira de Pareto tracejada (FP32) / pontilhada (INT8); um rótulo por modelo, posicionado sobre o ponto mais alto do par. Hachura indica elegibilidade arquitetural a Winograd (convolução 3×3 de *stride* 1 densa presente), não o traço medido de acionamento, mesma convenção da Figura 4.

Tabela IV
ROBUSTEZ DE QUANTIZAÇÃO: ΔQAT POR FAMÍLIA DE MODELO

Família	**Modelo**	**Top-1 FP32**	**Top-1 INT8**	**ΔQAT (p.p.)**
Compensação com 3×3	AlexNetResidual	48,01%	47,27%	−0,74
	AlexNetBottleneck	**44,62%**	**44,54%**	**−0,08**
	AlexNetDepthwiseSep	44,39%	41,47%	−2,92
	AlexNetFire	43,98%	44,30%	+0,33
	AlexNetFactorized	42,89%	42,60%	−0,29
	AlexNetGroupConv	29,18%	27,71%	−1,47
	Média	*42,18%*	*41,32%*	*−0,86*
Restrição ingênua	AlexNetSmallKernel	45,84%	36,16%	−9,67
	AlexNetStacked	45,22%	42,31%	−2,91
	AlexNet3x3-GAP	40,32%	37,02%	−3,29
	AlexNetMixed	38,74%	37,99%	−0,75
	AlexNet3x3-FC	35,79%	36,19%	+0,40
	AlexNet2x2-GAP	33,15%	30,82%	−2,33
	AlexNet2x2-FC	30,53%	31,54%	+1,01
	Média	*38,51%*	*36,00%*	*−2,51*
Baseline irrestrito	AlexNetTV (pré-trein.)	32,89%	31,90%	−0,98

Com relação a métodos adicionais de compressão além de INT8 padrão, testamos precisão mista INT4/INT8 e investigamos *weight-sharing* pós-hoc. A precisão mista atribui bits por camada por sensibilidade: os pesos de cada camada quantizável são quantizados de forma simulada para INT4 (por canal de saída), uma camada de cada vez; mede-se o erro quadrático médio (EQM) dos *logits* resultantes contra o *baseline* FP32 num único *batch* de calibração, e as camadas são ordenadas por essa sensibilidade — as 30% mais sensíveis permanecem em INT8, o restante vai para INT4, seguido de *fine-tuning*. A Figura 6 mostra que métodos abaixo de INT4 (ternária, INT2 e binária, todas com QAT) sofrem quedas de

22 a 37 p.p., enquanto INT4 QAT permanece viável (queda <2 p.p.) para famílias de compensação. Entre os métodos com retreino, a precisão mista INT4/INT8 oferece o melhor equilíbrio entre compressão e acurácia — não a maior taxa de compressão isoladamente, já que os métodos sub-INT4 mais agressivos comprimem mais (15,42× a 29,70×) às custas das quedas de 22 a 37 p.p. já citadas: a precisão mista mantém 7,05× de compressão com acurácia média de 44,52%, um ganho médio de 0,19 p.p. sobre o FP32 dos mesmos 3 modelos.

Três ressalvas limitam a aplicabilidade imediata desses resultados: (i) INT4 PTQ sem retreino colapsa (25,3%) porque a calibração de quantização com um único *batch* não captura a distribuição real de ativações — o retreino (QAT) é crítico, elevando a acurácia média de 25,3% para 43,3% ($1,7\times$; nos 3 modelos com as duas medições, 27,7%→43,3%); (ii) INT4 em hardware real permanece fora do escopo: o backend de quantização usado neste estudo (fbgemm) prioriza INT8, com suporte INT4 limitado; FPGA/aceleradores móveis suportam-no de forma inconsistente — esta medição é teórica; (iii) a compatibilidade com aceleração Winograd não é verificada — restrições de INT4 podem exigir reformatação de peso ou descontinuar a densidade que Winograd requer.

1) Nota adicional: viabilidade de poda estruturada: Como verificação adicional e preliminar (não um resultado central deste relatório), testamos poda estruturada de canais inteiros (não pesos individuais) no `AlexNetBottleneck`, razão 0,4, seleção por norma L1 [11]: 385.000→207.399 parâmetros (−46,1%; 53,9% dos parâmetros originais retidos), mantendo toda convolução remanescente densa (não agrupada, elegível à aceleração Winograd por construção). Sem *fine-tuning* após a poda a acurácia colapsa (top-1 0,50%), como esperado — este é apenas um teste de que a poda preserva essa densidade, não um resultado de acurácia competitivo. A literatura de poda estruturada por norma L1 mostra que *fine-tuning* pós-poda recupera acurácia próxima à do modelo original [11], o que motiva tratar essa recuperação como trabalho futuro plausível, não apenas como esperança.

2) Nota adicional: margem de compressão por compartilhamento de pesos: Em paralelo à poda, investigamos o potencial de compressão adicional além de INT8 puro, medindo quanto espaço ainda existe entre a entropia real dos pesos e a largura de bits nominal. Para o `AlexNetFire`, os pesos INT8 reais ocupam apenas 7,19 bits/peso de entropia de Shannon (contra 8,00 nominais), sugerindo margem para métodos de quantização com perda, como o compartilhamento de pesos (*weight-sharing*). Aplicamos *k-means* clustering — o passo de codebook do Deep Compression [12] — em 16/32/64 *clusters* (correspondendo a 4/5/6 bits nominais), e comparamos com o que o gzip já obtém: comprimir o *checkpoint* INT8 real em disco rende apenas 1,18× (0,553→0,469 MB), enquanto os três *codebooks* de *k-means* superam essa marca, chegando a ∼2,2× (0,245 MB, só pesos, valor teórico) em 16 *clusters*. É uma medição teórica, não uma avaliação de acurácia real pós-*clustering* nem uma mudança efetiva no formato de checkpoint — apenas um sinal de que vale a pena construir um *pipeline* real de compartilhamento de pesos (ver Trabalhos Futuros).

F. Eixo 6: Transferência para Predição Densa

Os cinco eixos anteriores tratam exclusivamente de classificação. Esta seção pergunta se a compensação estrutural leve (*bottleneck*, *Fire*) e sua robustez à quantização observadas em classificação transferem para predição densa — detecção de objetos (SSD [17]) e segmentação semântica (DeepLabV3 [18]) — usando os três mesmos *backbones* (`AlexNetBottleneck`, `AlexNetFire`, `AlexNetTV`) sobre PASCAL VOC [19] (detecção: *trainval* 2007+2012, teste 2007; segmentação: *train*/*val* 2012). As métricas usadas são **mAP** (*mean Average Precision*) [19], a área média sob a curva precisão-recall por classe em detecção — resume acerto de localização (IoU entre caixa predita e caixa-verdade) e de classificação num único número —, e **mIoU** (*mean Intersection over Union*) [20], a sobreposição média entre máscara predita e máscara-verdade por classe em segmentação.

Uma investigação preliminar revelou recall de âncora (fração de caixas-verdade cobertas por algum *anchor box* do gerador de caixas-âncora padrão do detector) de apenas 0,76–0,80 para os três *backbones*, bem abaixo do limiar de 95% considerado aceitável — causado por um erro de indexação nas camadas de extração de características (produzindo níveis de pirâmide duplicados) e por uma interpolação linear de escalas de âncora mal ajustada à distribuição real de tamanhos de caixa do VOC. Corrigidos os índices de *tap* e substituídas as escalas por valores explícitos ajustados a percentis da distribuição real, o recall sobe para 0,991/0,991/0,932 (*bottleneck*/*fire*/`AlexNetTV`) a 512px; os resultados abaixo usam exclusivamente execuções pós-correção. A quantização, em ambas as tarefas, incide apenas sobre o *backbone* — a *head* SSD/DeepLabV3 permanece em FP32.

As tabelas desta seção reportam três estágios, e a distinção entre os dois últimos importa para lê-las. A coluna **QAT** é o modelo em *quantization-aware training*: os pesos e ativações passam por *fake quantization* (arredondamento para a grade INT8 seguido de *dequantização*), mas a aritmética ainda é de ponto flutuante e a avaliação roda em GPU — é a acurácia que a quantização *simulada* prevê, medida no melhor *checkpoint* do *fine-tuning* de QAT. A coluna **INT8** é o modelo efetivamente convertido (quantização estática, *backend* fbgemm): pesos e ativações passam a inteiros de 8 bits, os *kernels* são de fato inteiros e a inferência é obrigatoriamente em CPU, avaliada uma única vez sobre o mesmo conjunto de validação. A diferença QAT→INT8 isola, portanto, o erro residual da conversão real frente à simulação — pequena quando a calibração dos observadores é fiel, e o motivo de o valor INT8 poder ficar tanto ligeiramente abaixo quanto ligeiramente acima do QAT. Já a diferença FP32→INT8 é a medida de robustez à quantização usada no restante do relatório. A mesma convenção vale para as Tabelas V, VI e XII.

A Tabela V resume os resultados de detecção (variante treinada do zero; variantes com *backbone* pré-treinado em Tiny ImageNet-200 seguem padrão semelhante — pratica-

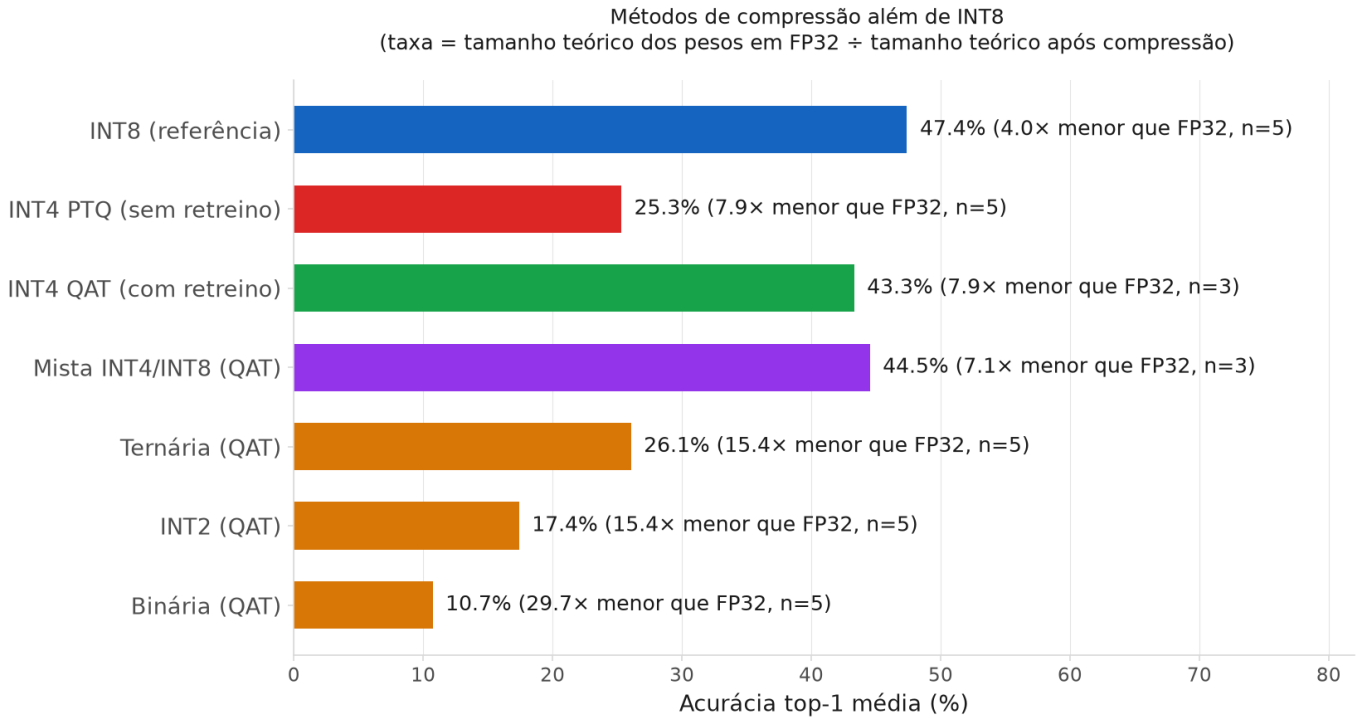


Figura 6. Acurácia top-1 média por método de compressão além de INT8, com taxa de compressão (tamanho teórico dos pesos em FP32 dividido pelo tamanho teórico após compressão) e número de modelos (n) entre parênteses. INT4 PTQ (sem retreino) colapsa para 25,3%; retreinar sob o mesmo INT4 (QAT) recupera para 43,3% — o retreino, não a largura de bits em si, é o que preserva a acurácia. As duas médias cobrem conjuntos de modelos diferentes (n=5 vs. n=3); restrita aos 3 modelos com as duas medições, a recuperação vai de 27,7% para 43,3%. Precisão mista INT4/INT8 (QAT) supera até o INT4 QAT uniforme com taxa de compressão comparável. Métodos abaixo de INT4 (ternária, INT2 e binária, todas com QAT) degradam progressivamente.

Tabela V
EIXO 6 — DETECÇÃO (SSD, VOC, BACKBONE DO ZERO). O MAP É A MÉDIA SOBRE IOU 0,5–0,95 E O TAMANHO É O DO MODELO REAL IMPLANTADO (TAMANHO REAL MEDIDO). OS VALORES DE ALEXNETFIRE MARCADOS COM ∼ USAM O TAMANHO DE CHECKPOINT BRUTO, NÃO MEDIDO DESSA FORMA NAQUELAS EXECUÇÕES. O MAP@0,5 DESTES MESMOS MODELOS, NA CONVENÇÃO DA LITERATURA, ESTÁ NA TABELA VII.

Backbone	Estágio	mAP (%)	Tam. (MB)
AlexNetBottleneck	FP32	**20,98**	5,76
	QAT	**21,00**	5,94
	INT8	**20,85**	3,14
AlexNetFire	FP32	8,94	6,26
	QAT	6,38	∼6,65
	INT8	6,40	∼3,47
AlexNetTV	FP32	16,94	4,10
	QAT	14,73	4,17
	INT8	14,50	2,22

Tabela VI
EIXO 6 — SEGMENTAÇÃO (DEEPLABV3, VOC 2012). MESMA CONVENÇÃO DA TABELA V: TAMANHO É O DO MODELO REAL IMPLANTADO (TAMANHO REAL MEDIDO).

Backbone	Estágio	mIoU (%)	Tam. (MB)
AlexNetBottleneck	FP32	17,84	12,13
	QAT	17,90	12,24
	INT8	**17,91**	11,19
AlexNetFire	FP32	17,03	12,63
	QAT	17,12	12,74
	INT8	17,05	11,31
AlexNetTV	FP32	16,34	10,26
	QAT	16,33	10,28
	INT8	16,21	9,32

mente não mudam para `AlexNetBottleneck` (ligeiramente pior nos três estágios), mas recuperam parte da queda de QAT/INT8 para `AlexNetFire` e `AlexNetTV`; Tabela XII). A Tabela VI resume segmentação.

A compensação de kernel pequeno transfere apenas parcialmente, e o mecanismo importa. `AlexNetBottleneck` lidera detecção (mAP 20,98%) e mantém sobre o *backbone* de kernel irrestrito `AlexNetTV` (16,94%) a mesma vantagem que tinha em classificação. Já `AlexNetFire`, que praticamente empata com `AlexNetBottleneck` em classificação (43,98% vs. 44,62% top-1, Eixo 1), cai para 8,94% — menos da metade. Paridade em classificação não implica paridade em localização.

A robustez de quantização transfere em segmentação: a deriva de mIoU FP32→INT8 fica em torno de 0,13 p.p. ou menos nos três *backbones*, reproduzindo o achado "quantização quase não custa acurácia" do Eixo 5. Em

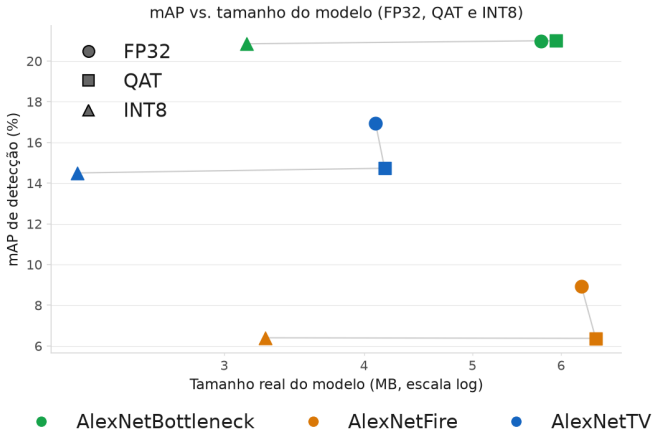


Figura 7. Detecção: mAP vs. tamanho real do modelo (tamanho medido, que exclui os pesos do classificador pré-treinado nunca usados no *forward pass* de detecção) nos estágios FP32, QAT e INT8 dos três *backbones*; uma linha cinza liga os estágios de cada *backbone*.

detecção o quadro é mais ruidoso — `AlexNetBottleneck` permanece plano (−0,14 p.p. de mAP), enquanto `AlexNetFire` e `AlexNetTV` perdem ∼2,4–2,5 p.p. `AlexNetBottleneck` é o único *backbone* cuja robustez se confirma sem ressalvas nas duas tarefas.

A sensibilidade a campo receptivo também depende da tarefa. Normalizando o mAP de detecção pela acurácia top-1 de classificação de cada *backbone* obtém-se 0,470 (*bottleneck*), 0,203 (*fire*) e 0,515 (`AlexNetTV`). `AlexNetFire` é o único caso de "pior em localização do que em classificação"; `AlexNetTV`, apesar de ser o classificador mais fraco dos três, é o menos penalizado pela troca de tarefa.

A *head* de detecção domina a latência: a do *pipeline* completo (*backbone*+*head* SSD, entrada 512×512) é 9,9×–15,0× maior que a do *backbone* isolado medida no Eixo 3 (entrada 64×64). A *head* e a resolução muito maior reintroduzem uma estrutura de latência que a comparação de kernel isolada no *backbone* não captura, diluindo qualquer sinal de aceleração Winograd no *pipeline* completo.

Um achado adicional e não previsto: o tamanho real do modelo (tamanho real medido, que exclui pesos do classificador pré-treinado original nunca usados no *forward pass* de detecção ou segmentação) inverte o *ranking* de eficiência de tamanho aparente. O *checkpoint* bruto de `AlexNetTV` chega a 223 MB — a rede classificadora completa, majoritariamente não utilizada. Seu tamanho real, porém, é de apenas 4,10 MB, menor que os 5,76 MB de `AlexNetBottleneck`, o que o torna o *backbone* mais eficiente em tamanho real nas duas tarefas — um resultado que o tamanho bruto do *checkpoint* escondia por completo.

Comparação com a literatura. Uma ressalva de métrica precede qualquer comparação: o mAP da Tabela V é a média sobre 10 limiares de IoU (0,50–0,95, convenção COCO), enquanto toda a literatura de detecção citada aqui reporta mAP@IoU=0,5 de limiar único (convenção PASCAL VOC). O mesmo treino já calcula esse valor de limiar único; é ele que a Tabela VII usa. Em segmentação não há discrepância: o mIoU (*macro* sobre 21 classes, descartando pixels de fronteira não rotulados) já segue a convenção padrão da literatura.

Duas diferenças de protocolo, essas sim, permanecem. Os *backbones* são pequenos (0,39–0,52M parâmetros) e, nas execuções usadas nestas duas tabelas, treinados *do zero* — sem pré-treino algum, enquanto quase toda a literatura citada parte de *backbones* pré-treinados em ImageNet (os modelos de segmentação da Tabela VIII são, além disso, pré-treinados em COCO). A exceção é `Tiny-DSOD`, também treinado do zero sobre VOC07+12 [26] — justamente a referência mais forte da Tabela VII, o que torna essa comparação específica a mais justa das cinco. A variante pré-treinada deste estudo (Tabela XII) usa Tiny ImageNet-200 (64×64, 200 classes), não ImageNet. A segmentação usa o *split* original de VOC2012 (1.464 imagens de treino) a 256px, sem o aumento via anotações SBD adotado por praticamente toda a literatura da área (∼10.582 imagens, 480–513px de *crop*). O orçamento de épocas, em compensação, deixou de ser uma diferença: após a correção do recall de âncora a detecção passou a treinar com até 1000 épocas e *early stopping* (paciência 50) sobre VOC07+12 *trainval*, com melhor época entre 549 e 927 (Tabela XII) — no mesmo patamar ou acima das ∼240 épocas-equivalente do SSD original. As Tabelas VII–VIII comparam os três *backbones* com referências eficientes por acurácia por megabyte de modelo real implantado (tamanho real medido) — eixo mais justo que a acurácia bruta, e mais alinhado à motivação de eficiência deste projeto.

Em detecção, os três *backbones* superam com folga os modelos densos clássicos, que nunca miraram tamanho de implantação: `AlexNetBottleneck` INT8 é ∼21× mais eficiente que SSD300. Mais informativo é o confronto com os *designs* mobile-first, que perseguem exatamente esse objetivo — `AlexNetBottleneck` e `AlexNetTV` em INT8 superam MobileNet-SSD, Pelee e Fire SSD por $4,6$–$5,6\times$, e ficam a ∼1,25× de `Tiny-DSOD`. O *design* dedicado ainda vence, mas por margem estreita.

Em segmentação a vantagem sobre os modelos densos se mantém (∼15× sobre FCN-8s, ∼4,7× sobre DeepLabv3/ResNet-101), mas a distância para os mobile-first é bem maior que em detecção: DeepLabv3+MobileNetV2 é $2,4\times$ mais eficiente em mIoU/MB que o melhor ponto deste estudo na variante com ASPP, e $5,1\times$ na variante sem ASPP (metade do tamanho por 0,4 p.p. de mIoU). A diferença de dado (VOC2012 sem aumento SBD, 256px) parece pesar mais nesta tarefa do que pesou em detecção. `AlexNetFire` é o pior dos três nas duas tarefas, reforçando o achado de que o mecanismo Fire não transfere bem para predição densa.

Limitações deste eixo. Qualquer estatística de correlação aqui usa $n \leq 3$ *backbones* e tem poder estatístico desprezível — leia-se como descritivo, não como evidência. O recall de âncora de `AlexNetTV` (0,932) permanece abaixo da meta de 0,95, aceito como uma limitação estrutural de sua pirâmide de características nativamente mais grosseira, não perseguido adiante (o recall abaixo do limiar foi aceito manualmente para essa execução). O *checkpoint* QAT/INT8 de `AlexNetFire`

Tabela VII
DETECÇÃO: EFICIÊNCIA ACURÁCIA/TAMANHO VS. LITERATURA (VOC, MAP@IOU=0,5 POR MB DE MODELO REAL; CONVENÇÃO DA LITERATURA CITADA, DIFERENTE DO MAP DA TABELA V)

Modelo	**Tam. (MB)**	**mAP@0,5**	**mAP@0,5/MB**
SSD300 (VGG16) [17]	∼105	77,2%	0,73
Fire SSD [27]	∼26	70,7%	2,72
MobileNet-SSD [25], [26]	22,0	68,0%	3,09
Pelee [25]	21,7	70,9%	3,26
Tiny-DSOD [26]	3,8	72,1%	**18,97**
`AlexNetFire` FP32	6,26	29,27%	4,68
`AlexNetBottleneck` FP32	5,76	47,46%	8,24
`AlexNetTV` FP32	4,10	37,67%	9,19
`AlexNetFire` INT8†	∼3,5	22,48%	∼6,49
`AlexNetTV` INT8	2,22	33,09%	14,92
`AlexNetBottleneck` INT8	3,14	47,83%	**15,23**

Tabela VIII
SEGMENTAÇÃO: EFICIÊNCIA ACURÁCIA/TAMANHO VS. LITERATURA (VOC2012, MIOU POR MB DE MODELO REAL)

Modelo	**Tam. (MB)**	**mIoU**	**mIoU/MB**
FCN-8s (VGG16) [20]	∼536	62,2%	0,116
DeepLabv3 (ResNet-101) [18]	∼233	85,7%	0,368
DeepLabv3+MobileNetV2 [6]	∼18,1	75,7%	4,19
idem, sem ASPP	∼8,4	75,3%	**8,92**
`AlexNetFire` FP32	12,63	17,03%	1,35
`AlexNetBottleneck` FP32	12,13	17,84%	1,47
`AlexNetTV` FP32	10,26	16,34%	1,59
`AlexNetFire` INT8	11,31	17,05%	1,51
`AlexNetBottleneck` INT8	11,19	17,91%	1,60
`AlexNetTV` INT8	9,32	16,21%	**1,74**

antecede uma correção do operador de concatenação quantizada do módulo *Fire*, então carrega sob o código atual apenas ignorando parâmetros ausentes (40 *buffers* de observador de quantização ausentes, todos escalares de poucos *bytes* cada — sem relação com peso de modelo real); com essa ressalva, o tamanho real medido foi recalculado normalmente para os dois estágios (Figura 7).

As comparações com a literatura carregam cinco ressalvas próprias. Os tamanhos de modelo das Tabelas VII–VIII são estimados a partir da contagem de parâmetros de cada artigo (params × 4 bytes, FP32) quando o tamanho de *checkpoint* não é informado — aproximação, não tamanho de arquivo real. Para as três referências de segmentação as contagens vêm todas da Tabela 7 de [6] (58,16M para o ResNet-101; 4,52M e 2,11M para as duas variantes MobileNetV2), de modo que as três linhas são medidas na mesma implementação. A linha marcada com † na Tabela VII usa o tamanho de *checkpoint* bruto, não o tamanho real medido, que não foi calculado naquela execução. O estágio QAT não entra nestas duas tabelas: é um *checkpoint* FP32 com quantização simulada, de tamanho praticamente igual ao do FP32 e que não é o artefato implantado, então uma razão acurácia/MB para ele não teria referente prático — as métricas de QAT estão nas Tabelas V–VI. O mIoU de 85,7% do DeepLabv3/ResNet-101 vem do *test set* oficial (via servidor de submissão), contra o *split val* usado aqui e por DeepLabv3+MobileNetV2 — são *splits* distintos, não diretamente comparáveis. E nenhuma referência mobile-first citada reporta variante INT8 no artigo original, então a comparação confronta o FP32 delas com o INT8 (mais compacto) destes modelos; em FP32 puro, `AlexNetBottleneck`/`AlexNetTV` ficam em 8,2–9,2 mAP@0,5/MB — ainda acima de MobileNet-SSD, Pelee e Fire SSD, e a ∼2× de `Tiny-DSOD`. Por fim, o mAP@0,5/22,0MB de "MobileNet-SSD" não vem do artigo original do MobileNetV1 [24] — que só reporta detecção SSD no COCO, não no PASCAL VOC — mas do *baseline* de mesmo nome reportado nas próprias comparações de Pelee e Tiny-DSOD, daí a citação dupla na tabela.

G. Eixo 7: Atenção Local Eficiente

Esta seção pergunta se atenção local em janelas (*windowed self-attention*, ao estilo Swin [22]) reproduz o perfil de acurácia/eficiência/robustez-à-quantização das CNNs de kernel pequeno deste estudo, e se ela é, por construção, incompatível com aceleração Winograd independentemente do tamanho de janela.

Sete modelos testam essas hipóteses (Tabela IX). A família `swin_pico_{w2,w4,w8}` varre o tamanho de janela $\in \{2, 4, 8\}$ — o análogo em atenção da restrição de kernel do Eixo 1. `hybrid_bottleneck_swin` antepõe um *stem* CNN de compensação aos estágios de atenção. `swin_pico_poolmixer` troca a atenção por um *pooling mixer* simples, controlando se o ganho vem da atenção real ou apenas da arquitetura ao redor dela. E o par `vit_tiny`/`deit_tiny` usa atenção global ao estilo ViT [21], com e sem destilação DeiT [23] de um professor `MobileNetV2` congelado.

Como não há caminho INT8 estável para LayerNorm/*softmax*/atenção neste *pipeline eager-mode* de QAT baseado em fbgemm, essas camadas permanecem em FP32 na conversão — apenas Linear/Conv vão para INT8, mesma exclusão seletiva já usada para BatchNorm neste estudo.

O tamanho de janela reproduz a curva de restrição de kernel (Figura 8). A acurácia top-1 cresce monotonicamente com a janela tanto em FP32 (32,95%→33,53%→36,75% para janelas 2/4/8) quanto em INT8 (33,25%→33,89%→36,99%): restringir o campo receptivo por camada custa acurácia também neste paradigma.

O *stem* CNN ajuda frente à atenção pura: o híbrido supera a Swin de janela máxima (`swin_pico_w8`) por +3,19 p.p. em INT8 a tamanho semelhante (0,542 vs. 0,803 MB). Mas não fecha a distância até as CNNs de compensação a tamanho equivalente — `AlexNetFireBypass` ocupa quase o mesmo espaço INT8 (∼0,55 MB) e atinge 49,86% contra 40,18% do híbrido.

Na robustez à quantização, o resultado contraria a hipótese inicial (de que excluir LayerNorm e atenção do QAT causaria perda sob INT8), mas não de forma uniforme. Cinco dos sete modelos ganham acurácia sob INT8 (de +0,24 a +2,10

Tabela IX
EIXO 7 — MODELOS DE ATENÇÃO LOCAL: PARÂMETROS E ACURÁCIA

Modelo	Params (M)	MACs (M)	Tam. FP32 (MB)	Tam. INT8 (MB)	Top-1 FP32 (%)	Top-1 INT8 (%)	ΔQAT (p.p.)
swin_pico_w2 (janela 2)	0,32	30,9	1,25	0,67	32,95	33,25	+0,31
swin_pico_w4 (janela 4)	0,32	31,8	1,26	0,68	33,53	33,89	+0,36
swin_pico_w8 (janela 8)	0,32	35,3	1,38	0,80	36,75	36,99	+0,24
swin_pico_poolmixer	0,23	21,2	0,89	0,30	30,98	30,76	−0,22
hybrid_bottleneck_swin	0,27	30,3	1,05	0,54	40,23	40,18	−0,06
vit_tiny	2,76	185,5	10,55	5,66	40,35	42,44	+2,10
deit_tiny	2,76	185,5	**10,55**	**5,66**	**46,38**	**47,66**	+1,28
vit_tiny_convstem†	2,93	222,0	11,20	5,58	48,00	49,29	+1,29
swin_pico_convstem†	0,36	59,3	1,42	0,72	41,87	45,66	+3,79

† *stem* com camada 3×3 de *stride* 1 acrescentada (elegível a Winograd $F(2 \times 2, 3 \times 3)$) — ver discussão ao final desta seção.

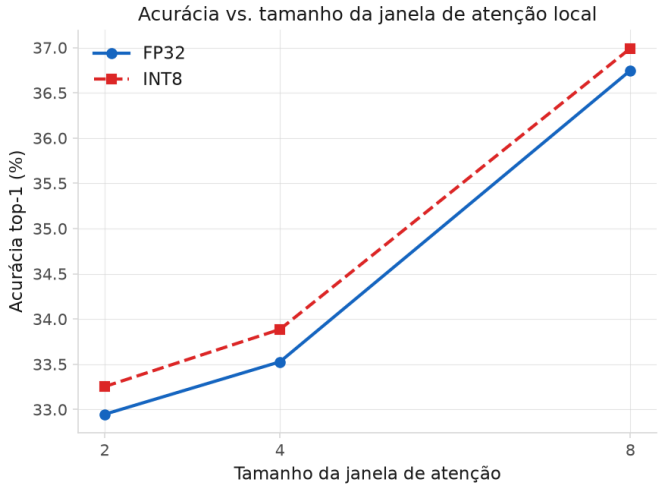


Figura 8. Acurácia top-1 (FP32 e INT8) vs. tamanho de janela de atenção local $\in \{2, 4, 8\}$ — análogo, em atenção, da curva de restrição de kernel do Eixo 1.

p.p.: os três `swin_pico_{w2,w4,w8}`, `vit_tiny` e `deit_tiny`), enquanto `swin_pico_poolmixer` (−0,22 p.p.) e `hybrid_bottleneck_swin` (−0,06 p.p., praticamente nulo) caem, ainda que pouco (coluna ΔQAT, Tabela IX). Os ganhos da família `swin_pico` (+0,24 a +0,36 p.p.) são modestos, próximos ao de `AlexNetFire` (+0,33 p.p., Eixo 1); só `vit_tiny`/`deit_tiny` mostram ganhos maiores. Uma leitura plausível para os ganhos que persistem é que, ao manter LayerNorm/atenção em FP32 e converter apenas as camadas Linear/Conv, o *fine-tuning* de QAT funciona como regularização adicional sobre a fração convertida do modelo — mas o mecanismo exato permanece especulativo, e não explica por que dois dos sete modelos não mostram esse efeito.

O custo da exclusão aparece no tamanho, não na acurácia. A robustez acima é medida em acurácia; olhando o *tamanho*, a mesma exclusão que preserva a acurácia cobra um preço claro. Manter LayerNorm/atenção em FP32 significa que uma fração grande dos pesos nunca converte, e a razão de compressão FP32→INT8 mede isso diretamente: os seis modelos com atenção comprimem apenas 1,72–1,94×, contra 3,08–3,99× (mediana 3,92×) das CNNs das Fases 1–4/9, que ficam próximas do teto teórico de 4× de 32 para 8 bits. A fração de parâmetros excluídos explica a diferença: 28,3–32,4% nos seis modelos com atenção.

O `swin_pico_poolmixer` funciona como controle deste efeito. Por substituir a atenção em janela por *pooling*, quase nada nele é excluído do QAT — 0,8% dos parâmetros — e ele comprime 2,91×, cerca de 1,5× melhor que seus irmãos com atenção, apesar de ser o menor modelo do eixo. A ordenação segue a fração excluída, não a arquitetura em si. Os valores absolutos ficam abaixo do que essa fração sozinha preveria (4/(1+3f), com f a fração excluída), porque escalas e *zero-points* por canal somam um custo fixo que pesa proporcionalmente mais nos modelos menores — é o que separa os 2,91× medidos do `poolmixer` dos ∼3,9× que sua fração excluída sugeriria.

A leitura conjunta é que a atenção local é penalizada duas vezes no perfil de implantação visado por este trabalho: não aciona *kernel* Winograd (por ausência de convoluções 3×3 de *stride* 1) e, sob este *pipeline* de QAT, converte para INT8 apenas parcialmente. Nenhuma das duas penalidades aparece na acurácia — que é onde a hipótese inicial esperava encontrá-las. A segunda, porém, depende da restrição de implementação descrita no início desta seção (sem caminho INT8 para LayerNorm/*softmax*), não de uma propriedade intrínseca da atenção: um *pipeline* com esse suporte fecharia a distância de compressão sem tocar na arquitetura.

A destilação DeiT confirma a hipótese: `deit_tiny` supera `vit_tiny` (mesma arquitetura, mesmos hiperparâmetros, só a função de perda muda) por +6,04 p.p. em FP32 e +5,22 p.p. em INT8. O ganho de destilar de um professor `MobileNetV2` congelado sobrevive à quantização e faz de `deit_tiny` o modelo mais forte entre os sete originais deste eixo — superado apenas por `vit_tiny_convstem` (48,00%/49,29%, Tabela IX), a variante com camada 3×3 no *stem* discutida adiante.

Nenhum dos sete modelos originais aciona Winograd em nenhuma configuração testada (traço de acionamento medido

como falso em todos). O resultado, porém, já era previsível pela arquitetura: nenhum deles contém convolução 3×3 de *stride* 1 — a única estrutura que aciona Winograd $F(2\times2, 3\times3)$. As únicas convoluções presentes são os *patch embeddings* (4×4 em `swin_pico`, 8×8 em `vit_tiny`/`deit_tiny`, ambos com *stride* igual ao *kernel*) e, no híbrido, um *stem bottleneck* de *stride* 2. A medição confirma a expectativa, mas não isola a atenção como causa: CNNs densas que *contêm* essa estrutura — `AlexNetBottleneck` e `ResNet18`, medidas no próprio Eixo 3 — também não acionam Winograd em nível de modelo completo, de modo que a ausência de traço não distingue "não tem convolução elegível" de "tem, mas o cuDNN escolheu outro algoritmo".

As duas linhas marcadas com † na Tabela IX preenchem essa lacuna ao nível arquitetural: `vit_tiny_convstem` e `swin_pico_convstem` repetem `vit_tiny` e `swin_pico` (janela 4), mas com uma camada 3×3 de *stride* 1 acrescentada ao *stem* — a mesma estrutura densa elegível a Winograd das CNNs deste estudo. Ambas ganham acurácia por uma margem larga sobre o par sem essa camada (+7,65/+6,85 p.p. FP32/INT8 para `vit_tiny_convstem`; +8,34/+11,77 p.p. para `swin_pico_convstem`, Figura 9), reforçando o achado já visto em `hybrid_bottleneck_swin` de que um *stem* CNN ajuda frente à atenção pura.

O rastreamento de *kernel* dessas duas variantes, feito na mesma GPU do Eixo 3, separa elegibilidade de acionamento: `vit_tiny_convstem` *aciona* Winograd — o *trace* registra dois *kernels* de nome Winograd (`generateWinogradTilesKernel` e `scudnn_winograd_128x128`), de forma estável em execuções repetidas —, mas `swin_pico_convstem` não, apesar de ter a mesma estrutura elegível. As duas camadas 3×3 de *stride* 1 rodam na mesma resolução (32×32) e diferem só em canais (32→64 contra 48→48); para a segunda, o cuDNN prefere um *implicit GEMM*. É o mesmo padrão do Eixo 3 — a elegibilidade arquitetural é necessária, não suficiente: quem decide é a heurística do *backend* para aquela configuração exata. Esses são os únicos dois modelos deste eixo com traço Winograd positivo ou sequer possível; nos sete originais o traço é negativo, como esperado pela ausência da estrutura.

O ganho não é grátis, e a comparação tem ressalvas. A camada acrescentada eleva os MACs de 31,8 para 59,3 M no `swin_pico` (+86%) e de 185,5 para 222,0 M no `vit_tiny` (+20%), com parâmetros de 0,32 para 0,36 M e de 2,76 para 2,93 M — parte do ganho pode vir de capacidade adicional, não da estrutura 3×3 em si. Os orçamentos de treino também diferem, e em direções opostas (Tabela XI): `vit_tiny_convstem` treinou 57 épocas FP32 contra 125 de `vit_tiny`, o que torna seu +7,65 p.p. uma estimativa conservadora; já `swin_pico_convstem` treinou 245 contra 143 de `swin_pico_w4`, de modo que parte dos +8,34 p.p. pode ser orçamento, não arquitetura. Somam-se a semente única e o hardware distinto (RTX 4060 local para as duas variantes †, RTX 4090 no PCAD para as demais).

Dois efeitos colaterais merecem registro. Primeiro, `swin_pico_convstem` tem ΔQAT de +3,79 p.p., o maior ganho sob INT8 de todo o eixo — entre os sete originais o maior era +2,10 p.p. de `vit_tiny` — coerente com o *stem* convolucional ser inteiramente quantizável, ao contrário dos blocos de atenção. Segundo, e apesar disso, o *stem* quase não alivia a penalidade de tamanho descrita acima: a razão de compressão FP32→INT8 sobe de ∼1,86× para 1,97× (`swin_pico_convstem`) e 2,01× (`vit_tiny_convstem`), ainda muito abaixo dos ∼3,9× das CNNs, porque a fração excluída do QAT continua dominada por LayerNorm/atenção.

Ainda assim, é a configuração de atenção que mais se aproxima das CNNs de compensação a tamanho equivalente: `swin_pico_convstem` atinge 45,66% INT8 a 0,72 MB contra 49,86% de `AlexNetFireBypass` a ∼0,55 MB — uma distância de ∼4,2 p.p., contra os ∼9,7 p.p. de `hybrid_bottleneck_swin` discutidos no início desta seção.

Uma ressalva vale para todo este eixo: `vit_tiny`/`deit_tiny` têm 2,76M parâmetros, cerca de 9× mais que a família `swin_pico` (0,23–0,32M), próximo do `VGG-Style` (2,41M, Eixo 1; o modelo de tamanho mais parecido no projeto) e 5–7× mais que as CNNs de compensação Pareto-ótimas (`AlexNetBottleneck` 0,39M, `AlexNetFire`/`AlexNetFireBypass` 0,52M) com as quais competem em acurácia. O resultado de destaque de `deit_tiny` (46–48% top-1) deve ser lido contra esse tamanho — competitivo com `VGG-Style` (51,81% FP32) a tamanho semelhante, mas não gratuito frente às CNNs de compensação.

Limitações deste eixo. A verificação de compatibilidade com Winograd mede apenas o modelo inteiro — não há, no código de perfilamento atual, discriminação por submódulo (*stem* vs. estágio de atenção) do tempo de dispositivo; a mesma ressalva do Eixo 3 sobre a instabilidade do nome do *kernel* cuDNN entre versões se aplica aqui, tornando a ausência de traço Winograd uma evidência fraca, não uma prova. A razão INT8/FP32 de tamanho reportada acima para a robustez de quantização usa o tamanho serializado dos *pesos* em ambos os lados (ver Seção II-C), não a contagem de parâmetros ponderada por precisão por módulo proposta originalmente para essa hipótese — uma proxy razoável de tamanho de implantação, mas não a mesma grandeza.

H. Síntese dos sete eixos

Em resumo, os sete eixos mostram:

1) *Custo da restrição de kernel (Eixo 1)*: nas comparações disponíveis, restringir kernels a 3×3 custou pouco ou nada (o único controle a geometria constante perdeu 1,0 p.p.), mas nenhuma delas isola o kernel de outras mudanças (geometria, pré-treino, duração do treino). A troca de *head* para GAP foi a mudança de efeito isolado mais claro (+4,5 p.p.).
2) *Mecanismos de compensação (Eixo 1)*: *bottleneck* e *Fire* recuperam mais 3,7–4,3 p.p. com muito menos

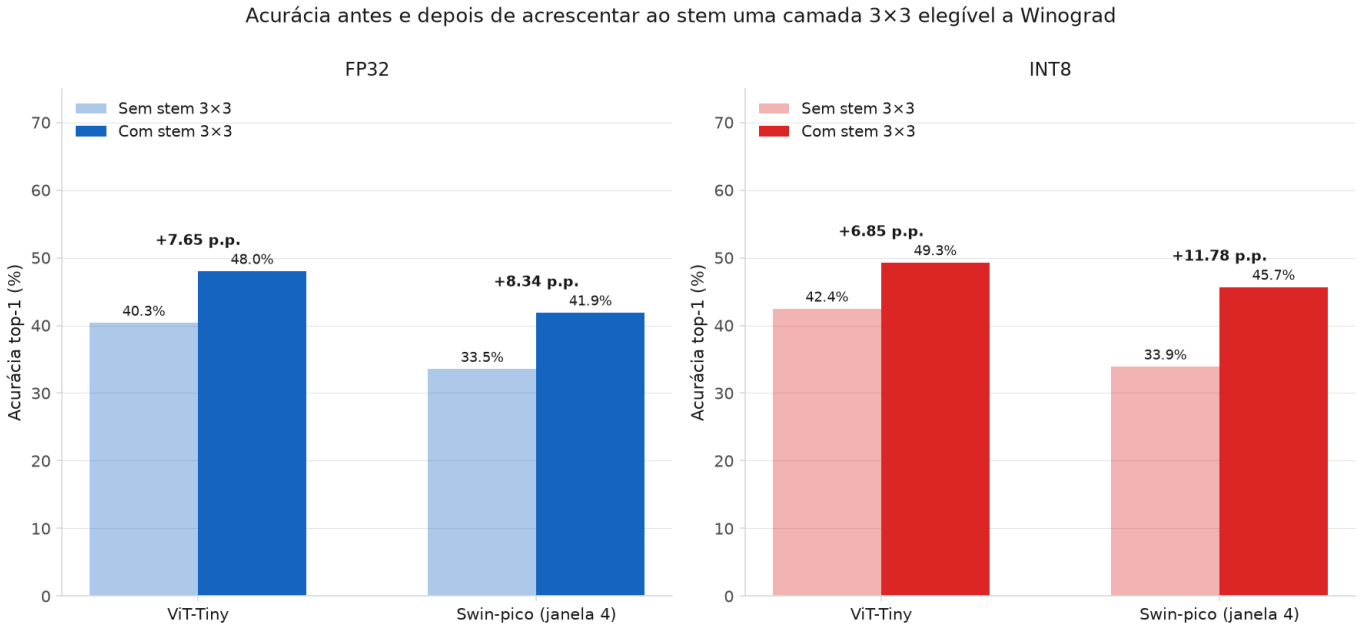


Figura 9. Ganho de acurácia top-1 (FP32 e INT8) ao acrescentar um *stem* 3×3 de *stride* 1 elegível a Winograd a `vit_tiny` e `swin_pico` (janela 4). O ganho não é grátis: a camada acrescenta MACs (+86% no `swin_pico`, +20% no `vit_tiny`) e os orçamentos de treino diferem entre os pares — ver texto.

parâmetros, mas só o *bottleneck* reduz também os MACs.

3) *Combinações de mecanismos (Eixo 2)*: Fire + Residual é a combinação mais efetiva (+5,81 p.p. acima do Fire de base); as combinações com *bottleneck* rendem pouco ou perdem acurácia.
4) *Validação em Hardware (Eixo 3)*: o cuDNN aciona Winograd para convoluções densas 3×3 em *batches* pequenos, nunca para *depthwise*; no regime *compute-bound*, *tensor cores* parecem competir com a vantagem de Winograd.
5) *Atalho de custo zero (Eixo 4)*: um atalho residual isolado supera, em FP32, o ganho da arquitetura híbrida completa (6,59 vs. 5,81 p.p. sobre o *Fire* de base), sem parâmetros extras; sob INT8, perde quase o mesmo que a híbrida (−0,71 vs. −0,59 p.p.).
6) *Robustez à quantização e compressão (Eixo 5)*: as variantes com compensação perdem em média 0,86 p.p. sob INT8, contra 2,51 p.p. das restrições ingênuas; abaixo de INT8, só INT4 com retreino e a precisão mista INT4/INT8 preservam a acurácia.
7) *Transferência para predição densa (Eixo 6)*: a compensação estrutural transfere para detecção e segmentação de forma específica por mecanismo, não universal — `AlexNetBottleneck` mantém sua vantagem, `AlexNetFire` colapsa.
8) *Atenção local eficiente (Eixo 7)*: atenção em janelas reproduz a curva de restrição de kernel e não aciona Winograd — por ausência de convoluções elegíveis, não por um efeito da atenção em si; contra a expectativa inicial, cinco dos sete modelos *ganham* acurácia sob INT8, e os outros dois caem apenas ligeiramente. As duas variantes com uma camada 3×3 acrescentada ao *stem* mostram que a elegibilidade a Winograd pode ser recuperada sem custo de acurácia — os dois ganham 6,9–11,8 p.p. sobre seus pares —, mas ao preço de mais MACs; e o rastreamento de *kernel* confirma acionamento real de Winograd em um deles (`vit_tiny_convstem`), não no outro, reiterando que elegibilidade não garante acionamento. A penalidade que resta é de *tamanho*: com LayerNorm/atenção fora do QAT, os modelos de atenção comprimem ∼1,7–2,0× contra ∼3,9× das CNNs.

IV. Discussão

Os modelos com compensação (bottleneck, Fire) quase não perdem acurácia sob quantização INT8 (−0,08 a +0,33 p.p.), enquanto as restrições ingênuas mais frágeis chegam a 3–10 p.p. de queda (`AlexNet3x3-GAP` −3,29, `AlexNetSmallKernel` −9,67; a média da família é −2,51 p.p., Tabela IV). Uma hipótese plausível é que as convoluções 1×1 do bottleneck/squeeze produzem ativações com distribuições mais regulares, facilitando a calibração de quantização. Um teste direto dessa hipótese permanece trabalho futuro.

No plano de hardware, medições em RTX 4060 mostram que *tensor cores* competem com Winograd em *batches* grandes (≥ 64) — uma limitação da plataforma de validação, não do princípio em si (Eixo 3 detalha a medição e suas ressalvas). Na mesma linha, MobileNetV2 e AlexNetDepthwiseSep têm kernels pequenos, mas nunca acionam um *kernel* Winograd nesta GPU/*backend* (∼51% menos GFLOP/s de mediana que modelos densos comparáveis). Isso sugere que arquiteturas efi-

cientes para dispositivos móveis podem necessitar estratégias de otimização diferentes daquelas propostas aqui, sem que isso implique incompatibilidade matemática entre Winograd e convoluções *depthwise* em geral.

Limitações. (i) A comparação com a baseline irrestrita (Tabela I) mistura vários fatores — tamanho de kernel, pré-treino ImageNet, geometria de *stride*/*pooling* (original vs. adaptada a 64×64), *dropout* e duração de treino (79 vs. 37 épocas) — que não são separáveis com os controles deste estudo. `AlexNet3x3-FC` iguala a *head* FC da baseline, mas nenhuma variante restrita é pré-treinada, usa a geometria original ou tem *dropout*, e o estudo não inclui uma AlexNet de kernels grandes na geometria adaptada; o efeito puro do kernel permanece confundido (Seção II-B). Da mesma forma, `AlexNetBottleneck` e `AlexNetFire` usam *BatchNorm* em todas as convoluções e lr 10^{-3}, enquanto `AlexNet3x3-GAP` não usa BN e treinou com lr 3×10^{-4}; no experimento suplementar da Fase 11 (protocolo distinto), BN sozinho valeu +2,8 a +4,6 p.p., da ordem dos +4,3 p.p. atribuídos ao *bottleneck*, de modo que o ganho da compensação estrutural não está separado do efeito de BN e de lr (a vantagem em parâmetros e MACs independe disso). Controles com o mesmo lr, com BN e no protocolo de 500 épocas da Fase 11 não constam deste relatório. Além disso, os valores absolutos são de 64×64 e não se extrapolam para 224×224. (ii) O modelo `AlexNetTV` foi treinado de forma independente em dois momentos do projeto (79 e 67 épocas), com resultados diferentes, especialmente em INT8 (31,90% vs. 26,28%). Este relatório padroniza na execução de 79 épocas (a mesma usada no perfilamento de hardware) em todas as tabelas, mas a diferença entre execuções é, em si, um indício de variância não desprezível entre treinos nominalmente idênticos. (iii) Toda a validação de hardware é de uma única GPU (RTX 4060 Laptop), em uma única sessão de coleta — sem repetição entre sessões, sem outra GPU, e sem qualquer medição em FPGA. O Eixo 3 já discute como isso limita a leitura dos testes estatísticos de latência. (iv) Os testes estatísticos de hardware operam com n pequeno (8–24 configurações), o que limita seu poder. A evidência mais confiável desta seção é o rastreamento direto do nome do *kernel*, não os testes de latência. (v) Nenhum dos resultados de acurácia usa múltiplas sementes de treino. Diferenças de poucos décimos de ponto percentual entre arquiteturas próximas não devem ser lidas como significativas.

V. Conclusão

Nas comparações deste estudo, restringir os kernels de uma CNN a 3×3, para torná-la compatível com aceleradores Winograd, custou pouco ou nada em acurácia (Eixo 1), contra a expectativa inicial do projeto: o único controle a geometria constante perdeu 1,0 p.p. Nenhuma dessas comparações, porém, isola o kernel de outras mudanças — a 64×64, a AlexNet original perde acurácia principalmente pela geometria, e as variantes restritas diferem dela também em pré-treino e duração do treino. O que de fato eleva a acurácia acima da baseline pré-treinada é a combinação com um *head* GAP e mecanismos de compensação leves — blocos *bottleneck*, módulos *Fire*, ou atalhos residuais isolados — nenhum dos quais reintroduz kernel $> 3 \times 3$.

Com apenas 0,385M parâmetros (1,51 MB) e 39,5M MACs — menos de um quarto dos MACs do módulo Fire (177,7M), a menor contagem de computação entre os mecanismos de compensação *densos* testados — o `AlexNetBottleneck` atinge 44,62% de acurácia top-1 em FP32 e mantém 44,54% após INT8 (queda de apenas 0,08 p.p., a menor perda registrada entre as CNNs deste estudo — alguns modelos chegam a *ganhar* acurácia sob INT8). Entre os mecanismos de compensação estrutural testados, é o único que reduz memória *e* computação simultaneamente frente à baseline restrita+GAP *sem* recorrer a convoluções *depthwise*: o `AlexNetDepthwiseSep` reduz ainda mais os dois (0,314M, 1,23 MB, 19,8M MACs), mas ao preço da elegibilidade a Winograd (Eixo 3) — distinção relevante para um acelerador cujo custo escala principalmente com número de multiplicações.

O experimento com o atalho residual isolado mostra que essa adição sozinha (zero parâmetros extras) supera, em FP32, o ganho de uma arquitetura híbrida bem mais elaborada da mesma família (6,59 p.p. sobre o *Fire* de base, contra 5,81 p.p. da híbrida completa): uma mudança estrutural simples chegou mais longe que a combinação de várias, ao menos em FP32. Uma ressalva crítica, porém: o `AlexNetFireBypass` usou muito mais épocas que o `AlexNetFinalFireResidual` (152 vs. 71, Apêndice B), então parte desse ganho pode vir do treino mais longo, e não só da arquitetura. Sob quantização, os dois perdem quantidades parecidas (−0,71 vs. −0,59 p.p.), e o atalho isolado continua 0,66 p.p. à frente em INT8.

A validação em hardware (RTX 4060, sessão única) usa rastreamento direto do nome do *kernel* CUDA e mostra que o cuDNN aciona um *kernel* Winograd para convoluções densas 3×3 em *batches* pequenos (1 e 8) — e nunca para convoluções *depthwise*, em nenhum tamanho de kernel testado. O teste estatístico de *scaling* de latência é, no entanto, significativo na direção oposta à esperada no regime *compute-bound* (*batch*=64), o mais relevante para um acelerador dedicado. Três achados limitam o escopo desta validação: (i) *tensor cores* parecem competir com a vantagem de Winograd justamente no regime *compute-bound*, tornando o ganho de latência inconsistente entre *batches*; (ii) convoluções *depthwise* nunca acionam Winograd nesta GPU/*backend*, ajudando a explicar por que MobileNetV2 não se beneficia como as arquiteturas densas; (iii) toda a medição vem de uma única GPU e uma única sessão de coleta. Um acelerador FPGA dedicado (motivação deste projeto) não possui *tensor cores* e, por construção, não sofreria a competição de (i) — mas essa é uma expectativa de *design*, não algo medido neste trabalho.

Estendendo essas descobertas além de classificação, a transferência para predição densa (Eixo 6) mostra que o benefício da compensação estrutural não é universal. Ele se mantém integralmente para `AlexNetBottleneck` em detecção e segmentação, mas colapsa para `AlexNetFire` — apesar de os dois mecanismos serem quase intercambiáveis em

classificação. Qual mecanismo específico foi escolhido importa mais para predição densa do que para classificação.

Já a investigação de atenção local (Eixo 7) confirma a expectativa estrutural do princípio deste projeto: nenhum dos sete modelos testados aciona Winograd em qualquer configuração, por ausência de convoluções elegíveis e não por um efeito da atenção em si, e a acurácia segue a mesma curva de restrição de campo receptivo observada para kernels convolucionais. O resultado de quantização, porém, é misto e mais fraco do que o esperado: em vez de perderem acurácia sob INT8, cinco dos sete modelos de atenção local *ganham* (`swin_pico_{w2,w4,w8}`, `vit_tiny`, `deit_tiny`), com ganhos modestos para a família `swin_pico` — comparáveis ao de `AlexNetFire` (+0,33 p.p.) — enquanto os outros dois (`swin_pico_poolmixer`, `hybrid_bottleneck_swin`) caem ligeiramente. Uma leitura plausível para os ganhos que persistem é que manter LayerNorm/atenção em FP32 durante o QAT evita um caminho de quantização instável e transforma o restante do *fine-tuning* em regularização adicional — mas o mecanismo não explica os dois casos que não ganham. A destilação DeiT é o mais forte dos sete modelos de atenção originais (46,38% FP32; só a variante `vit_tiny_convstem`, com uma camada 3×3 acrescentada ao *stem*, o supera com 48,00%), mas usa 5–7× o número de parâmetros das CNNs Pareto-ótimas deste projeto (2,76M vs. 0,39–0,52M): competitivo em acurácia, não em eficiência.

VI. Trabalhos Futuros

1) *Fine-tuning* após a poda estruturada: recuperar a acurácia do `AlexNetBottleneck` podado (46,1% menos parâmetros, 0,385M → 0,207M), mantendo a elegibilidade a Winograd por meio de convoluções densas (não agrupadas); a literatura de poda por norma L1 [11] sugere que essa recuperação é factível.
2) Compartilhamento de pesos com retreino: implementar um *pipeline* real de *weight-sharing* [12] com retreino, em contraste com a medição teórica de margem de compressão já feita neste estudo, explorando os 7,19 bits/peso efetivos dos pesos INT8.
3) Perfilamento de hardware estendido: medições de latência e potência em FPGA real ou acelerador dedicado, para verificar se a competição com *tensor cores* é um artefato da GPU.

As investigações de transferência para predição densa (Eixo 6 — detecção e segmentação) e de atenção local eficiente (Eixo 7) já estão concluídas, com resultados completos neste relatório (Seções III-F e III-G); as lacunas que restam em cada uma estão descritas nas respectivas "Limitações deste eixo". Esses dois eixos mostram que a abordagem de restringir o kernel e compensar com estrutura se estende, de forma específica por mecanismo, além da classificação (predição densa) e além das CNNs puras (atenção local).

Agradecimentos

Experimentos utilizaram recursos da infraestrutura PCAD (http://pcad.inf.ufrgs.br).

Referências

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[2] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, "Error analysis and improving the accuracy of Winograd convolution for deep neural networks," *ACM Transactions on Mathematical Software*, vol. 46, no. 4, Art. 37, 2020.

[4] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," *arXiv preprint arXiv:1602.07360*, 2016.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.

[7] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.

[8] D. Khudia, J. Huang, P. Basu, S. Deng, H. Liu, J. Park, and M. Smelyanskiy, "FBGEMM: Enabling high-performance low-precision deep learning inference," *arXiv preprint arXiv:2101.05615*, 2021.

[9] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," Stanford CS231N Report, 2015.

[10] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[11] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *International Conference on Learning Representations (ICLR)*, 2017.

[12] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.

[13] G. Li, R. Li, T. Li, T. Chen, M. Zhang, and H. Corporaal, "Algorithm-hardware co-design for accelerating depthwise separable CNNs," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 2, 2025.

[14] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[15] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 113–123.

[16] M. Lin, Q. Chen, and S. Yan, "Network in network," in *International Conference on Learning Representations (ICLR)*, 2014.

[17] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conference on Computer Vision (ECCV)*, 2016.

[18] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[19] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[20] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[21] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[22] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[23] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. International Conference on Machine Learning (ICML)*, 2021.

[24] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[25] R. J. Wang, X. Li, and C. X. Ling, "Pelee: A real-time object detection system on mobile devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[26] Y. Li, J. Li, W. Lin, and J. Li, "Tiny-DSOD: Lightweight object detection for resource-restricted usages," in *Proc. British Machine Vision Conference (BMVC)*, 2018.

[27] H. Liau, N. Yamini, and Y. Wong, "Fire SSD: Wide fire modules based single shot detector on edge device," *arXiv preprint arXiv:1806.05363*, 2018.

APÊNDICE

A. Diagramas de Arquitetura

As Figuras 10–13 mostram o diagrama de blocos dos quatro modelos que sustentam os Eixos 1, 2 e 4 — a fronteira de Pareto acurácia/tamanho/quantização, construída inteiramente com convoluções $1\times1/3\times3$ (nenhum kernel $>3\times3$) — com canais de entrada/saída, parâmetros e tipo de convolução (1×1 em cinza, 3×3 em verde) extraídos dos pesos reais. Dispostos na horizontal para caber dois por página; rótulos internos em inglês.

B. Dados de Treinamento

A Tabela X compara o protocolo padrão de hiperparâmetros, usado nas execuções locais, com os ajustes de orçamento aplicados nas execuções do PCAD; a taxa de aprendizado inicial é um hiperparâmetro por modelo, listada na Tabela XI. No QAT, a BatchNorm é congelada na época 3 e os observadores de quantização desativados na época 5, seguindo a prática recomendada por [7]. No PCAD, a paciência do *early stopping* varia por grupo: 20 para as híbridas do Eixo 2, a arquitetura híbrida final e `AlexNetFireBypass`; 50 para os modelos de atenção do Eixo 7, que também usam *weight decay* 0,05 no FP32 (receita de [23]) e *warmup* de 5 épocas — únicas exceções ao valor comum entre local e PCAD. O orçamento de detecção/segmentação do Eixo 6 segue um protocolo à parte (Tabela XII), fora do escopo da tabela abaixo.

A Tabela XI consolida, por modelo, taxa de aprendizado, épocas e tempo de treino por estágio (FP32/QAT) e acurácia final top-1/top-5 (FP32/INT8), agrupados como na Figura 1. Épocas/tempo de QAT vêm do mesmo resumo de execução que registra a acurácia FP32/INT8 já reportada, garantindo a mesma execução (ao contrário de *logs* de QAT residuais de uma execução *large-scale* superada, arquivados à parte e não usados aqui). A coluna de épocas é a contagem registrada no resumo de execução de cada modelo — a época do melhor *checkpoint* onde o resumo a separa da contagem total; para alguns estágios de QAT sem esse campo, ela vem do *log* e é a época de parada, não a do melhor *checkpoint*. O tempo usa o campo agregado de tempo total de treino, exceto nos modelos da Fase 2 — onde ele retorna 0 ou valores inconsistentes — para os quais soma-se a duração real por época. Marcado com $*$, `AlexNetSE` é exceção: seu resumo de execução registra zero épocas apesar do treino ter rodado (colapso, não ausência). O tempo total que ele registra, 297,9 s (≈4,9 min), dividido pelo tempo médio por época do mesmo resumo (48,8 s) dá 6 épocas — a única estimativa disponível, já que nem *checkpoint* nem *log* desse treino sobreviveram. Células "—" de QAT valem só para `MobileNetV2`/`ResNet18` pré-treinados (nunca passaram por QAT — *residual adds* sem operador de soma dedicado à quantização) e `AlexNetSE` (colapso). INT8 não é um estágio treinado (calibração estática em segundos), sem coluna de tempo. Marcados com †: *baselines* irrestritos, restrição ingênua, `AlexNetBottleneck`/`AlexNetFire` e as demais variantes de mecanismo único de "Tentativas de compensação" treinaram localmente (RTX 4060) — inferido pela ausência, no resumo de execução, do bloco de metadados de ambiente que só a *pipeline* do PCAD grava. Os demais modelos — híbridas do Eixo 2, arquitetura híbrida final e os sete do Eixo 7 — são atribuídos ao PCAD (RTX 4090), mas quatro deles (três híbridas do Eixo 2 e `AlexNetFinalFireResidual`) têm resumos de execução sem esse mesmo bloco de metadados, o que não confirma nem descarta a origem. Tempos entre os dois ambientes, e frente à latência do Eixo 3 (Seção II-D), não são comparáveis. O `AlexNetFireBypass` treinou por mais épocas que os demais (152 vs. ∼32–71 nas híbridas), daí seu tempo desproporcional. Os modelos do Eixo 7 compartilham uma única taxa de aprendizado/*weight decay* (5×10^{-4}/0,05, [23]) e, exceto `vit_tiny`/`deit_tiny` (via *notebook* dedicado, confirmados por seus *logs*), encerraram por *early stopping* (paciência 50) antes do teto de época. As variantes †`_convstem` (camada 3×3 elegível a Winograd acrescentada ao *stem*, Seção III-G) treinaram localmente seguindo a convenção geral.

C. Dados de Treinamento — Predição Densa (Eixo 6)

A Tabela XII lista, por *backbone*, tarefa e variante (do zero ou pré-treinado no Tiny ImageNet-200, Seção III-F — sem variante pré-treinada para segmentação, não suportada nesta tarefa), a época do melhor *checkpoint*, o tempo de treino FP32 e a métrica nos três estágios FP32/QAT/INT8 (mAP ou mIoU), todos no PCAD (RTX 4090). O orçamento de detecção foi ampliado para até 1000 épocas (paciência 50) após a correção do *bug* de recall de âncora, daí tempos da ordem de dias, bem maiores que os de classificação ou segmentação. O tempo tabulado é só do estágio FP32 (QAT tem orçamento próprio mais curto; INT8 não é treinado; ambos omitidos por brevidade — ver Seção III-F). A coluna de tempo é tempo de parede de execuções em um *cluster* compartilhado e **não deve ser lida como custo de treino de cada variante**. O tempo por época muda em degraus dentro de uma *mesma* execução — `AlexNetTV` do zero roda a ∼305 s/época nos 60% iniciais do treino, ∼520 s/época nos 20% seguintes e ∼865 s/época nos 5% finais; o `AlexNetBottleneck` pré-treinado sai de ∼160 s/época para ∼525 s/época nos últimos 30% — com modelo, configuração e dados idênticos, o que só pode vir de disputa por recurso no nó. Descontado esse efeito, o custo por época no patamar não disputado é essencialmente o mesmo com e sem pré-treino em dois dos três *backbones* (p10 do tempo de época, do zero vs. pré-treinado: 156 vs. 142 s para `AlexNetBottleneck` e 296 vs. 293 s para `AlexNetTV`) — como esperado, já que o pré-treino muda os pesos iniciais, não o custo de uma passagem. O `AlexNetFire` pré-treinado é a exceção: sua execução sobe monotonicamente do início ao fim (médias por quinto de 376 a 544 s/época) e não tem um patamar não disputado a isolar, de modo que seu p10 (329 s) fica acima do da execução do zero (281 s) sem que isso diga nada sobre a arquitetura. O que o pré-treino de fato altera é o número de épocas até o *early stopping*, e mesmo isso sem direção única: 760 vs. 600 épocas (`AlexNetBottleneck`), 647 vs. 823 (`AlexNetFire`) e 839 vs. 978 (`AlexNetTV`).

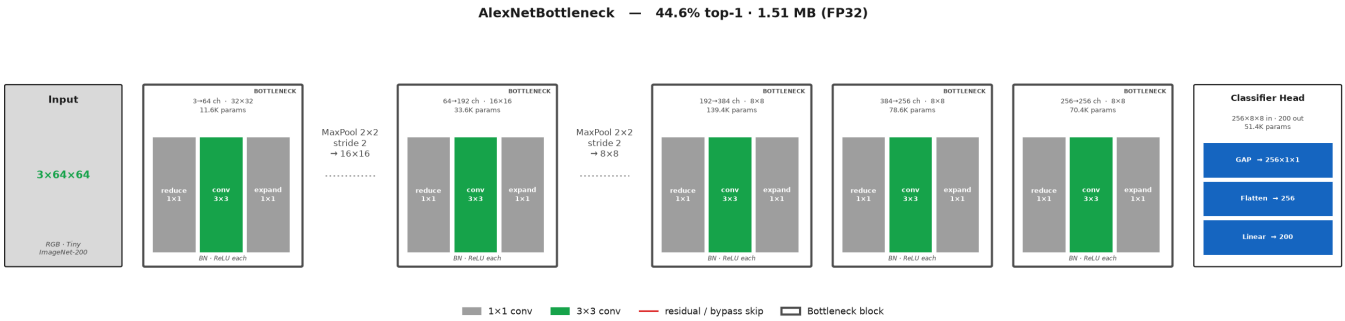


Figura 10. `AlexNetBottleneck`: bloco *reduce* 1×1 → *conv* 3×3 → *expand* 1×1 por estágio.

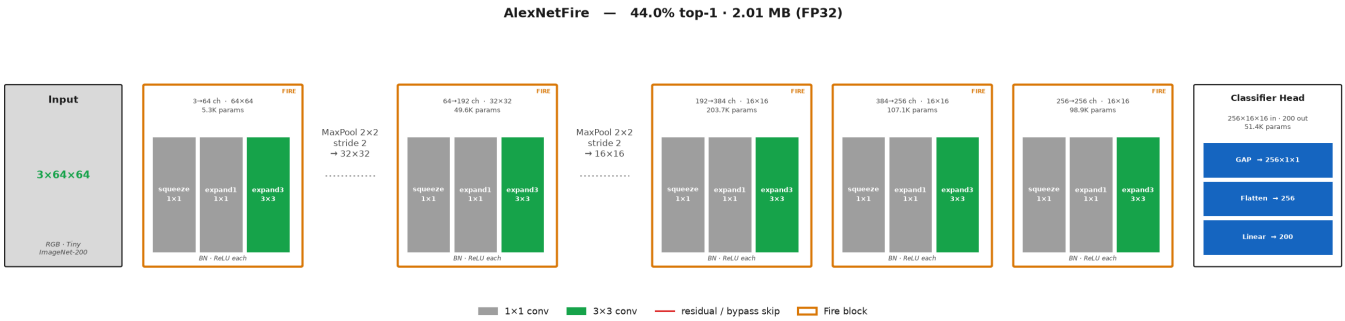


Figura 11. `AlexNetFire`: módulo *Fire* (*squeeze* 1×1 → *expand* 1×1/3×3 concatenados) por estágio.

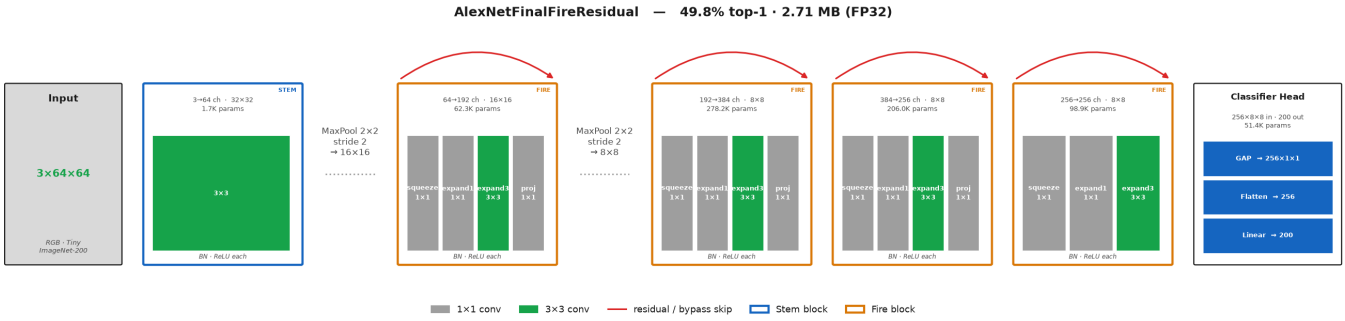


Figura 12. `AlexNetFinalFireResidual`: *stem* 3×3 *stride*-2 seguido de módulos *Fire* com atalho residual em cada estágio.

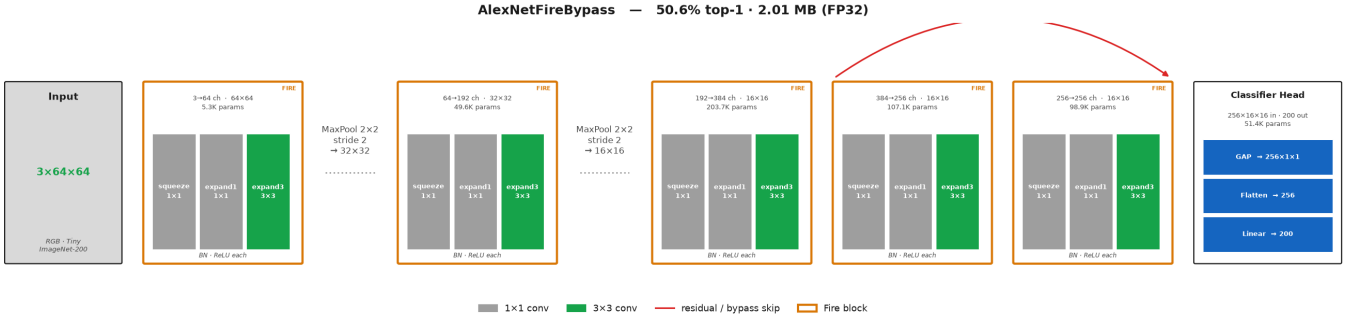


Figura 13. `AlexNetFireBypass`: `AlexNetFire` base com um atalho residual por identidade adicionado a cada estágio, sem nenhum parâmetro extra.

Tabela X
HIPERPARÂMETROS DE TREINAMENTO: PADRÃO LOCAL (RTX 4060) VS. AJUSTES DE ORÇAMENTO NO PCAD (RTX 4090)

Parâmetro	**Local (RTX 4060)**	**PCAD (RTX 4090)**
Otimizador	AdamW [14]	
Scheduler	*Cosine annealing* (T_max = épocas)	
Épocas (orçamento FP32)	100, paciência 5	até 1000, paciência 20–50
Épocas (orçamento QAT)	20	até 100
Weight decay (FP32 / QAT)	5×10^{-4} / 5×10^{-4}	5×10^{-4}–0,05 / 5×10^{-4}
Label smoothing	0,1	
Batch size	64	
AMP (*mixed precision*)	Ligado (FP32) / desligado (QAT)	
Split treino/validação	90/10 determinístico, seed 42	
Aumento de dados (treino)	*RandomResizedCrop* (0,7–1,0), *flip* horizontal, *RandomRotation* (15°), AutoAugment (política ImageNet) [15]	
Aumento de dados (validação)	*Resize* → *CenterCrop*	
Normalização	Média/desvio-padrão do ImageNet	
QAT: *freeze* BN / desativa *observers*	Época 3 / Época 5	
INT8: *backend* / dispositivo	fbgemm / CPU	

Tabela XI
TAXA DE APRENDIZADO, ÉPOCAS E TEMPO DE TREINO (FP32/QAT), TAMANHO DO MODELO (PESOS, FP32/INT8) E ACURÁCIA TOP-1/TOP-5 (FP32/INT8) POR MODELO, 32 DOS 35 DO ESTUDO (DEMAIS SÃO REEXECUÇÕES SUPERADAS). † TREINADO LOCALMENTE (RTX 4060); SEM †, PCAD (RTX 4090) — VER RESSALVA NO TEXTO ANTERIOR. * ESTIMADO, VER TEXTO.

Família	**Modelo**	**LR**	**Ép. FP32**	**Ép. QAT**	**T. FP32 (min)**	**T. QAT (min)**	**Tam. FP32 (MB)**	**Tam. INT8 (MB)**	**Top-1/Top-5 FP32 (%)**	**Top-1/Top-5 INT8 (%)**
Baselines irrestritos	AlexNetTV† (pré-trein.)	3×10^{-4}	79	2	63,2	6,9	220,58	55,33	32,89 / 58,22	31,90 / 57,24
	MobileNetV2† (pré-trein.)	1×10^{-4}	27	—	14,8	—	9,69	—	58,00 / 81,51	—
	ResNet18† (pré-trein.)	1×10^{-4}	15	—	7,1	—	43,07	—	53,91 / 77,80	—
	VGG-Style†	1×10^{-3}	62	6	30,5	8,5	9,21	2,38	51,81 / 75,88	51,19 / 75,52
Restrição ingênua	AlexNet3x3-FC†	3×10^{-4}	37	15	29,1	16,7	219,75	55,12	35,79 / 61,28	36,19 / 62,16
	AlexNet3x3-GAP†	3×10^{-4}	43	19	16,3	7,9	8,79	2,23	40,32 / 66,32	37,02 / 62,89
	AlexNet2x2-FC†	3×10^{-4}	28	4	14,2	6,2	134,98	33,93	30,53 / 56,03	31,54 / 57,15
	AlexNet2x2-GAP†	3×10^{-4}	44	16	15,5	6,3	4,02	1,04	33,15 / 58,95	30,82 / 56,09
	AlexNetMixed†	3×10^{-4}	45	19	15,7	7,3	6,68	1,71	38,74 / 64,83	37,99 / 63,80
	AlexNetStacked†	1×10^{-3}	92	13	88,8	26,5	230,77	57,94	45,22 / 70,96	42,31 / 68,35
	AlexNetSmallKernel†	3×10^{-4}	62	18	24,5	15,8	6,12	1,56	45,84 / 71,08	36,16 / 61,60
Tentativas de compensação	AlexNetBottleneck†	1×10^{-3}	76	4	29,6	5,0	1,51	0,43	44,62 / 71,04	44,54 / 71,01
	AlexNetFire†	1×10^{-3}	41	8	21,7	15,2	2,01	0,55	43,98 / 70,43	44,30 / 70,88
	AlexNetDepthwiseSep†	1×10^{-3}	95	6	38,2	4,8	1,23	0,40	44,39 / 69,45	41,47 / 66,99
	AlexNetFactorized†	3×10^{-4}	66	5	65,8	16,8	217,73	54,68	42,89 / 68,91	42,60 / 67,80
	AlexNetGroupConv†	1×10^{-3}	91	4	68,6	9,6	213,34	53,54	29,18 / 57,07	27,71 / 55,10
	AlexNetResidual†	3×10^{-4}	62	12	64,0	28,9	231,49	58,10	48,01 / 73,11	47,27 / 72,50
	AlexNetSE†	3×10^{-4}	6*	—	4,9*	—	219,91	—	0,50 / 2,50	—
	Bottleneck + Fire	1×10^{-3}	34	13	13,9	9,4	1,97	0,55	42,29 / 68,54	44,00 / 70,15
	Bottleneck + Residual	1×10^{-3}	32	9	15,6	9,4	2,24	0,64	45,10 / 71,15	45,98 / 72,33
	Depthwise + Fire	1×10^{-3}	70	3	27,3	3,3	1,85	0,51	43,46 / 69,02	42,79 / 68,95
Arquitetura híbrida final	Fire + Residual	1×10^{-3}	71	3	28,8	3,7	2,71	0,75	49,79 / 74,80	49,20 / 74,39
	AlexNetFireBypass	1×10^{-3}	152	50	86,2	27,8	2,01	0,55	50,57 / 74,90	49,86 / 74,56
Atenção local (Eixo 7)	swin_pico_w2	5×10^{-4}	146	99	156,1	107,3	1,25	0,67	32,95 / 59,97	33,25 / 60,45
	swin_pico_w4	5×10^{-4}	143	91	150,2	97,3	1,26	0,68	33,53 / 60,73	33,89 / 61,20
	swin_pico_w8	5×10^{-4}	144	80	80,7	44,8	1,38	0,80	36,75 / 63,51	36,99 / 63,93
	swin_pico_poolmixer	5×10^{-4}	143	99	80,6	55,5	0,89	0,30	30,98 / 57,60	30,76 / 57,84
	hybrid_bottleneck_swin	5×10^{-4}	144	96	85,9	56,7	1,05	0,54	40,23 / 66,65	40,18 / 66,35
	vit_tiny	5×10^{-4}	125	89	59,6	65,8	10,55	5,66	40,35 / 65,58	42,44 / 66,38
	deit_tiny	5×10^{-4}	212	62	108,4	46,1	10,55	5,66	46,38 / 72,09	47,66 / 71,34
	vit_tiny_convstem†	5×10^{-4}	57	32	24,3	28,0	11,20	5,58	48,00 / 72,32	49,29 / 72,46
	swin_pico_convstem†	5×10^{-4}	245	53	105,2	27,9	1,42	0,72	41,87 / 69,15	45,66 / 72,21

Tabela XII
EIXO 6 — DETECÇÃO E SEGMENTAÇÃO, DO ZERO VS. PRÉ-TREINADO: MELHOR ÉPOCA E TEMPO DE TREINO FP32, COM MÉTRICA NOS TRÊS ESTÁGIOS FP32/QAT/INT8 (MAP PARA DETECÇÃO, MIOU PARA SEGMENTAÇÃO, AMBAS EM %) POR BACKBONE. O TEMPO É DE PAREDE EM NÓ COMPARTILHADO E NÃO É COMPARÁVEL ENTRE EXECUÇÕES (VER TEXTO ANTERIOR).

Backbone	**Tarefa**	**Variante**	**Melhor época**	**Tempo (h)**	**FP32**	**QAT**	**INT8**
AlexNetBottleneck	Detecção	do zero	549	29,7	20,98%	21,00%	20,85%
AlexNetFire	Detecção	do zero	772	68,6	8,94%	6,38%	6,40%
AlexNetTV	Detecção	do zero	927	115,0	16,94%	14,73%	14,50%
AlexNetBottleneck	Detecção	pré-treinado	709	56,1	20,83%	20,15%	19,79%
AlexNetFire	Detecção	pré-treinado	596	88,4	8,93%	7,79%	7,54%
AlexNetTV	Detecção	pré-treinado	788	125,1	18,10%	17,24%	16,96%
AlexNetBottleneck	Segmentação	do zero	84	0,5	17,84%	17,90%	17,91%
AlexNetFire	Segmentação	do zero	154	1,1	17,03%	17,12%	17,05%
AlexNetTV	Segmentação	do zero	99	0,6	16,34%	16,33%	16,21%